

LLM & RAG 101

12 กันยายน 2569

คณะวิทยาศาสตร์ มหาวิทยาลัยศิลปากร

โดย ภาควัต นาควิจิตร, PhD

pakawat nk@gmail.com



เอกสารประกอบ
การอบรม



Labs: <https://github.com/chameleonTK/lms-rag-101>

Publicly Available

2019

- T5
- GPT-3
- GShard
- mT5
- Codex
- PanGu- α
- Anthropic
- WebGPT
- Ernie 3.0
- Gopher
- InstructGPT

2022

- ChatGPT
- OPT
- GLM
- Galatica
- BLOOM
- LaMDA
- AlphaCode
- Flan-T5

2023

- GPT-4
- LLaMA2
- YuLan-Chat
- ChatGLM
- Falcon
- MOSS
- PaLM2
- PanGu- Σ
- Bard
- LLaMA
- Deepseek
- InternLM
- Mistral
- Mixtral
- Qwen

2024

- Gemini 2.0
- Gemma-2
- GPT-4o
- InternLM2
- Qwen2
- DeepSeek-V2
- LLaMA3
- MiniCPM
- Gemma
- DeepSeek-R1
- Kimi K1.5
- Seed 1.5 / 1.6
- GPT-o3
- LLaDA
- GPT-4.5
- LLaMA4
- Qwen3
- Gemini 2.5
- Kimi K2
- Step 3
- GLM 4.5 / 4.6
- Gemma-3
- Claude 4.5
- Ring-1T / Ling-1T
- MiniMax M2
- GPT-5.1
- Gemini 3.0
- Grok 4.1
- DeepSeek-V3.2
- Mistral 3
- GPT-5.2
- MiMo-V2
- Seed 1.8
- Nanbeige4
- GLM 4.7
- MiniMax M2.1

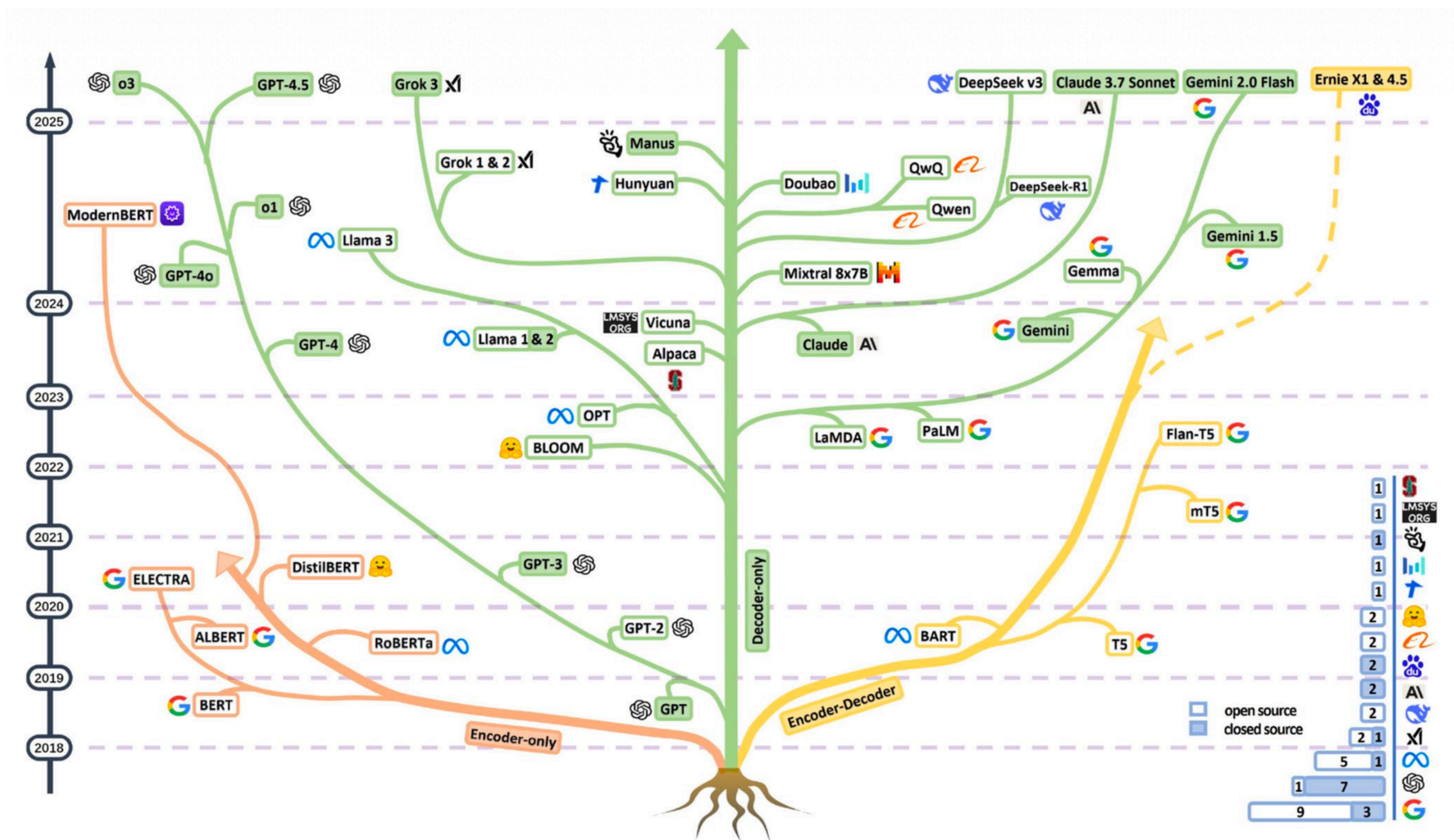
2025

- Claude 3.5
- Qwen2.5
- DeepSeek-V3
- YuLan-Mini
- Mixtral Large 2

2026

1-6, 7-11, 12

พื้นฐานร่วมกัน: Language Modeling



ทุกๆ โมเดลต่างพัฒนาขึ้นเพื่อ
แก้ปัญหาเดียวกัน คือ

ปัญหา Language Modeling

Quiz Time



โจทย์: จงเติมคำในช่องว่าง ??? ให้ถูกต้อง

1. อดทนจนกว่าจะ ????????
2. ชามเขียวคว่ำเข้า ชามขาว??????????
3. ใครอยากเป็นเศรษฐี ??? ???
4. แคลตาซอย 5 บาท ????????????

ปัญหา Language Modeling คืออะไร?

Language Modeling คือ ปัญหาในการทายคำถัดไป จากประโยคตั้งต้น
= Next-Token Prediction

ประโยคตั้งต้น

Kaniva Kaniva อาหารแมว

คำถัดไปที่โมเดลทำนาย

โซเดียมต่ำ และไม่เค็ม

ปัญหา Language Modeling คืออะไร?

ประโยคตั้งต้น

Kaniva Kaniva อาหารแมว

$245 + 21 =$

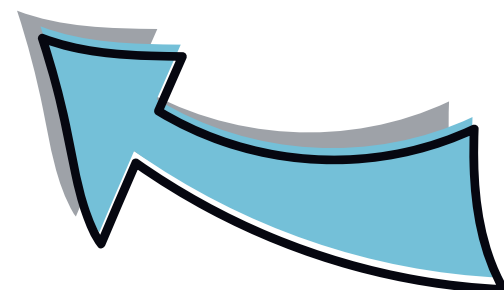
ประเทศไทยมีประชากรเท่าไร?

คำตอบที่โมเดลทำนาย

โซเดียมต่ำ และไม่เค็ม

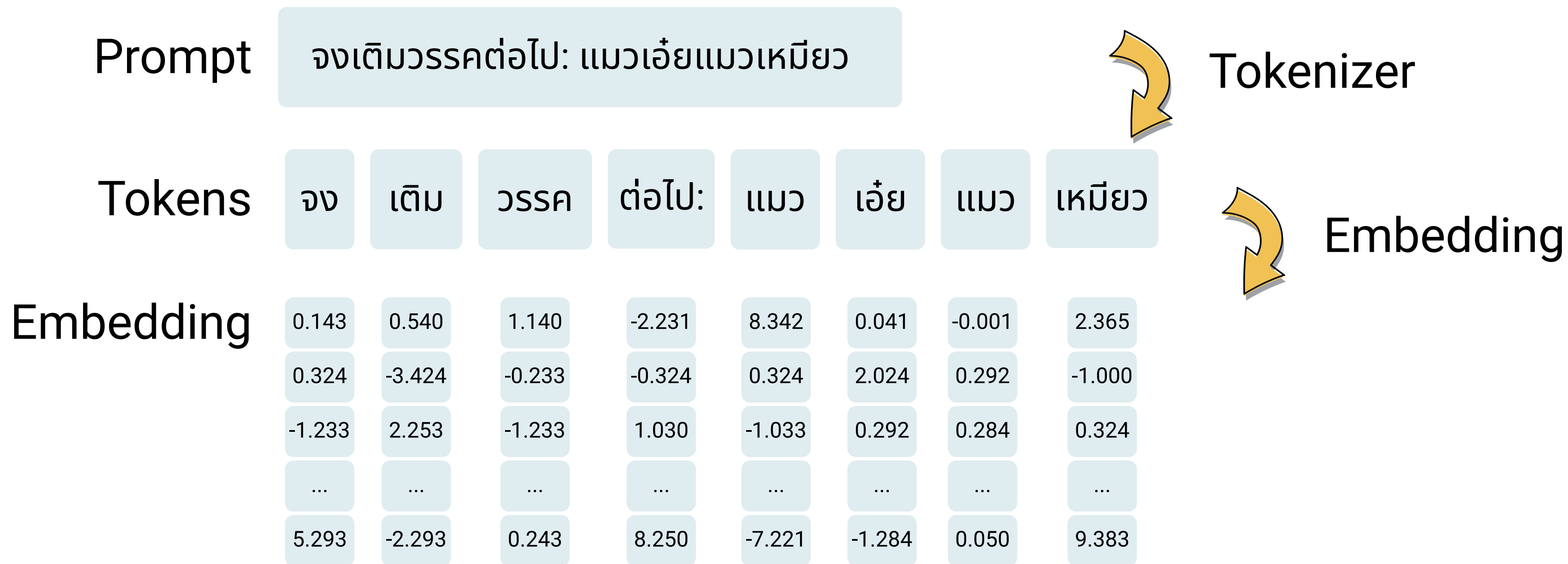
266

65.81 ล้านคน



ปัจจุบัน เรียกว่า **Prompt**

เกิดอะไรขึ้น เมื่อเรา **prompt** ไปยัง LLMs?



เกิดอะไรขึ้น เมื่อเรา **prompt** ไปยัง LLMs?

Embedding	จง	เต็ม	จssc	ต่อไป:	แมว	เอ๋ย	แมว	เหมียว
	0.143	0.540	1.140	-2.231	8.342	0.041	-0.001	2.365
	...	...	...	...	...	...	...	...
	5.293	-2.293	0.243	8.250	-7.221	-1.284	0.050	9.383

Multi-head self-attention layer

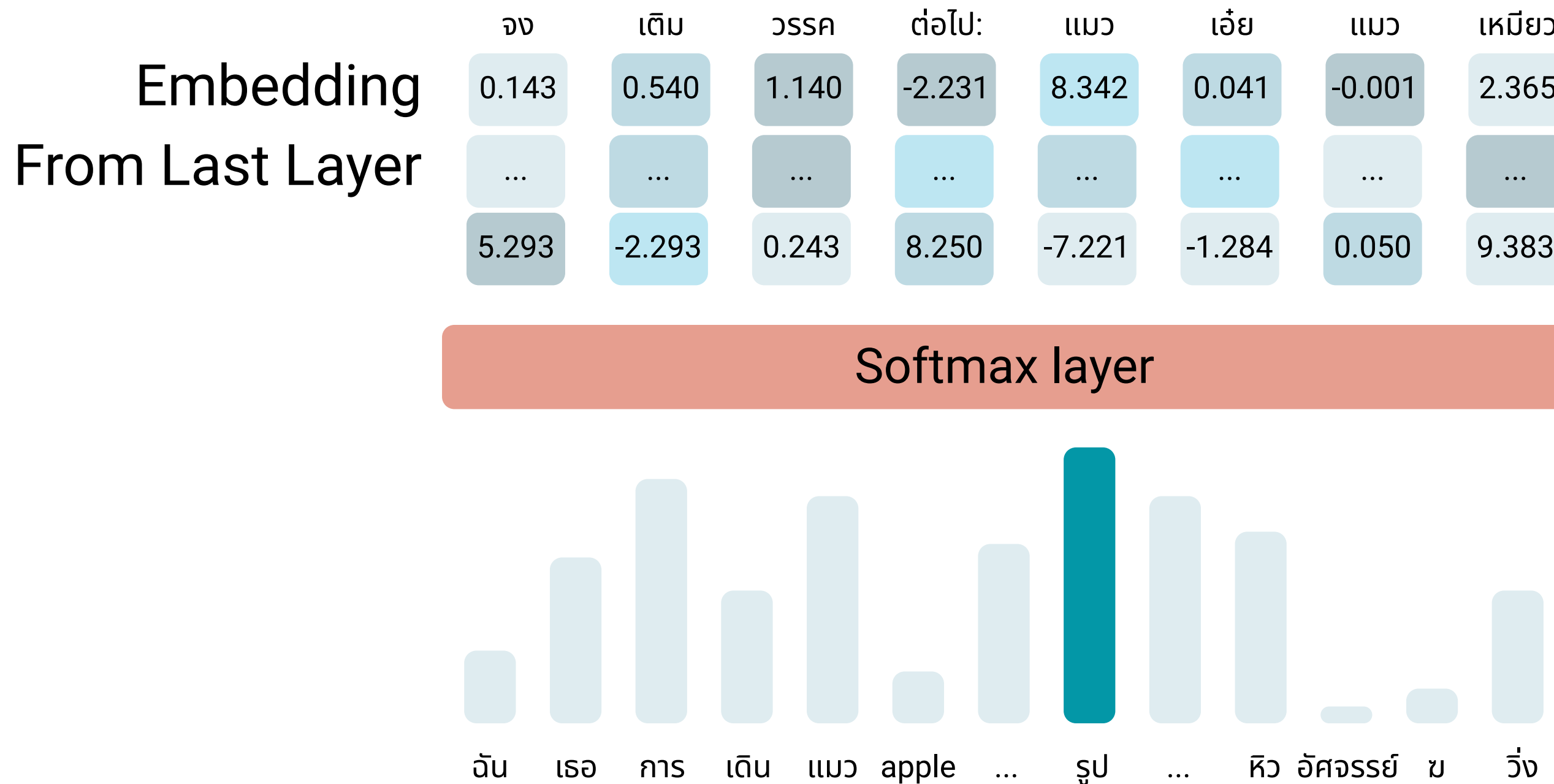
Feed-Forward Network

Embedding	0.143	0.540	1.140	-2.231	8.342	0.041	-0.001	2.365
	...	...	...	...	...	...	...	...
	5.293	-2.293	0.243	8.250	-7.221	-1.284	0.050	9.383



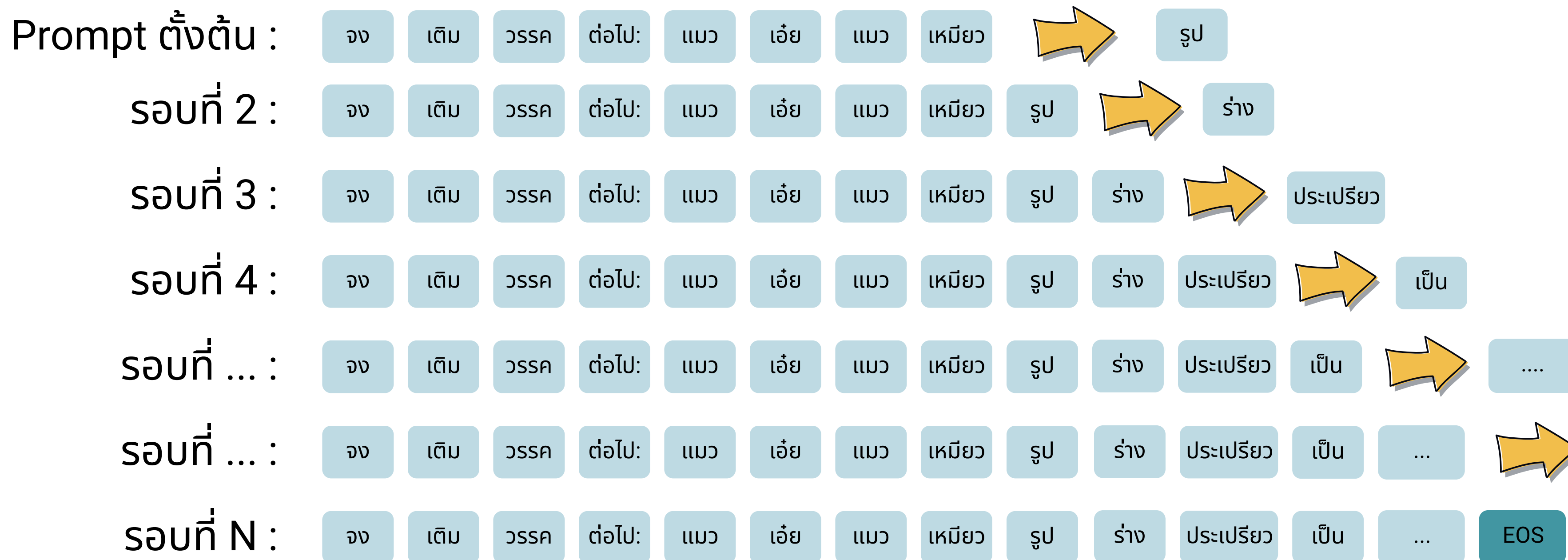
Transformer Layer
ทำแบบนี้ซ้ำๆ x64 ครั้ง
(Qwen3.5-27B)

เกิดอะไรขึ้น เมื่อเรา **prompt** ไปยัง LLMs?

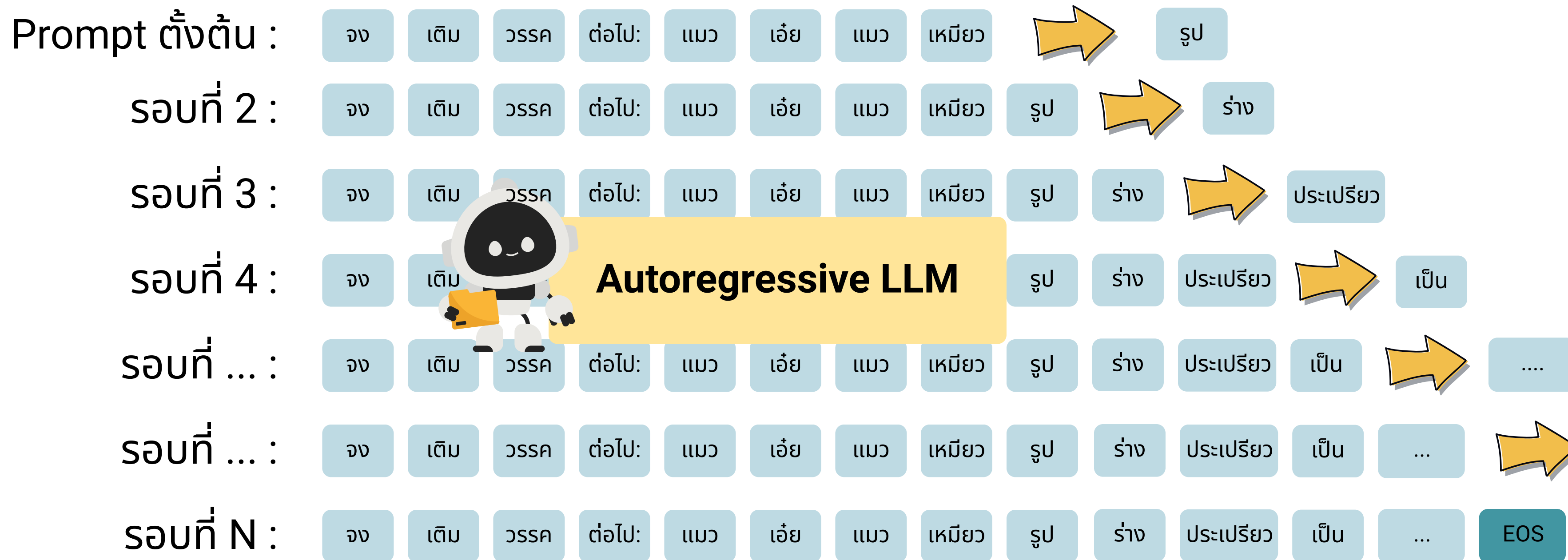


คำถัดไป = รูป

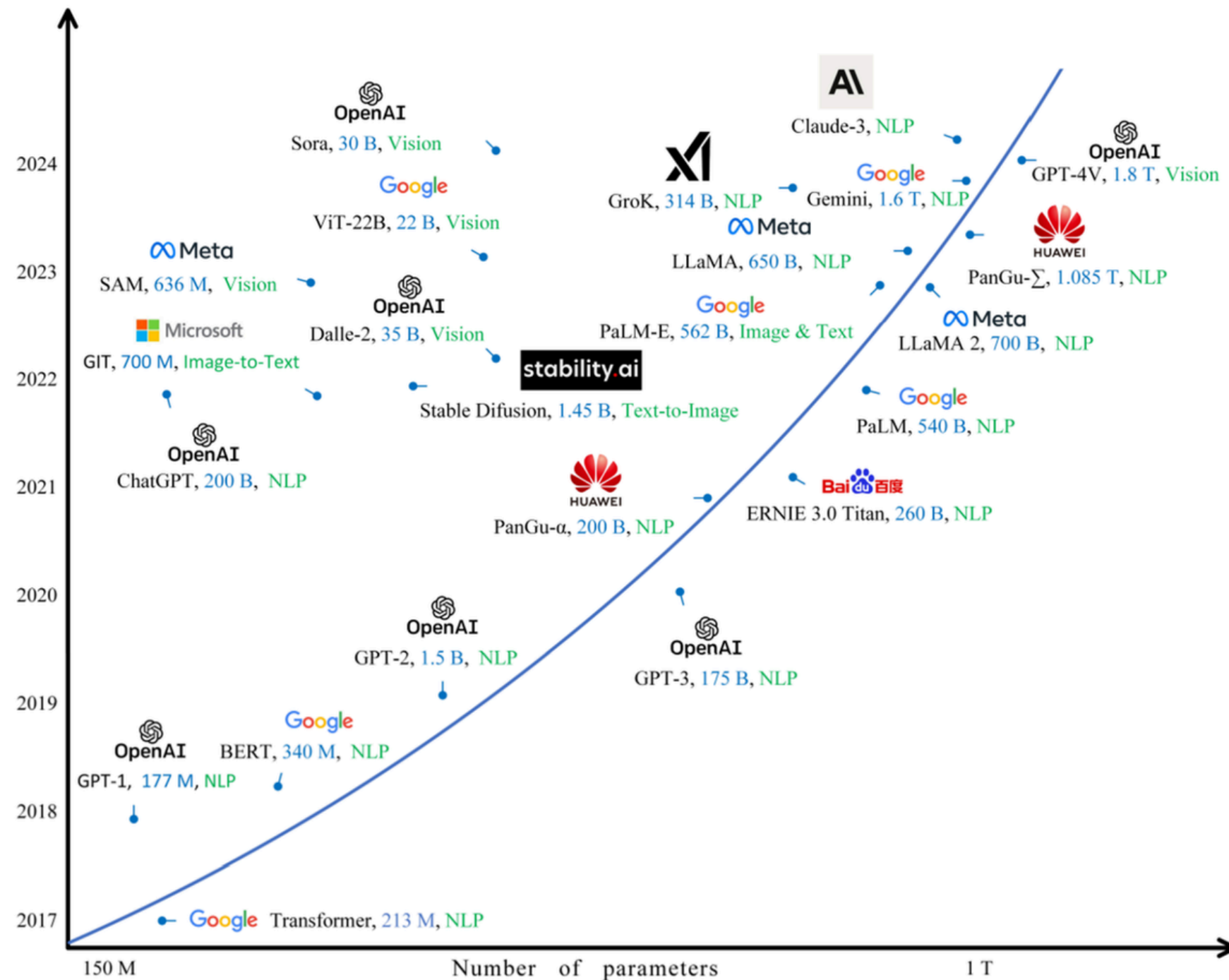
เกิดอะไรขึ้น เมื่อเรา **prompt** ไปยัง LLMs?



เกิดอะไรขึ้น เมื่อเรา **prompt** ไปยัง LLMs?

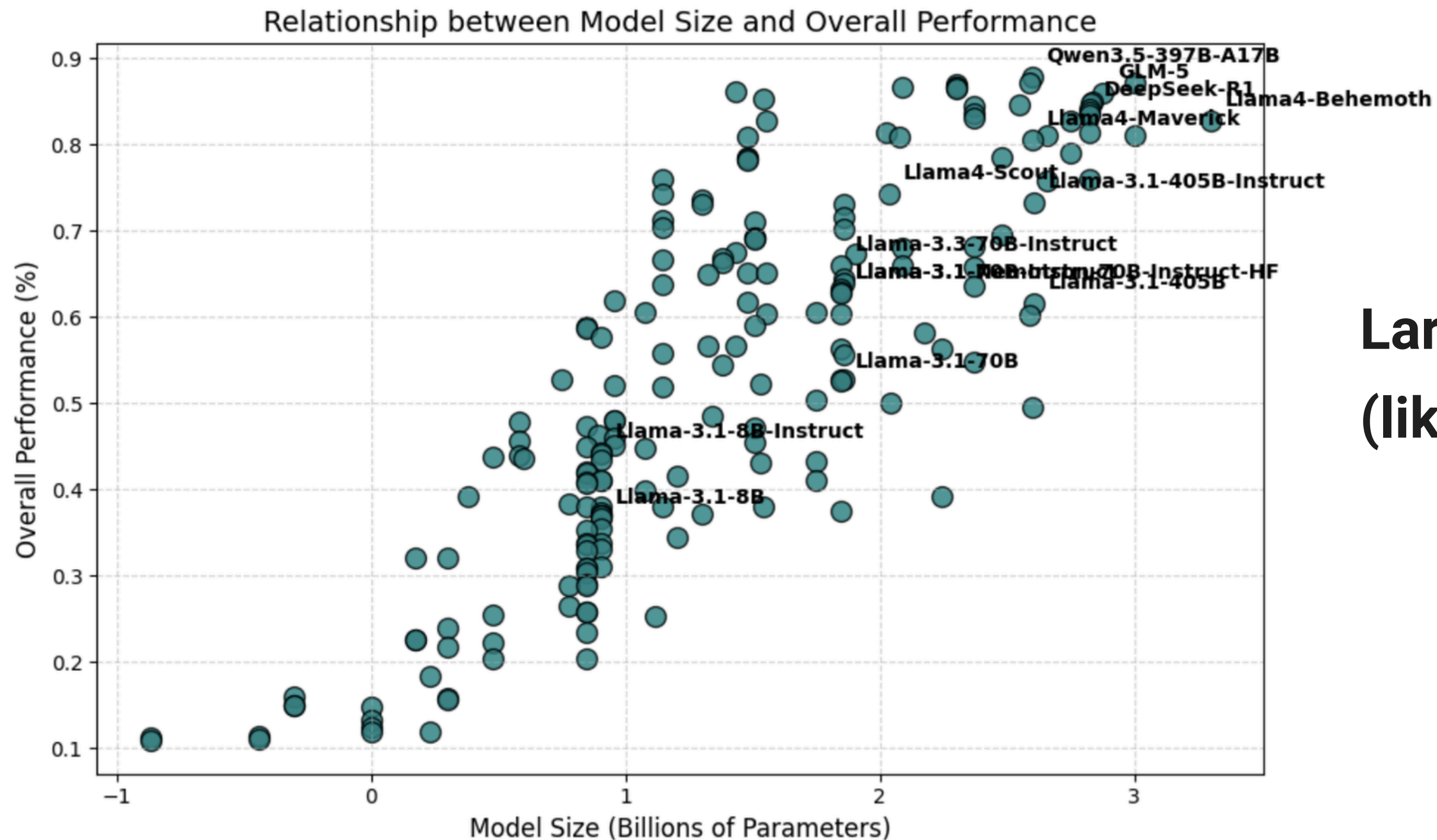


2-3 ปีที่ผ่านมา เกิดอะไรขึ้นกับ LM?



- โมเดลมีขนาดใหญ่ขึ้น จาก 150M เป็น 20B เป็น 1T
- ถูกสอนด้วยข้อมูลมากขึ้น
- ต้องใช้หน่วยประมวลผลขนาดใหญ่ขึ้น
- Training Cost ↑
- Inference Cost ↑
- Latency ↑
- memory use ↑

ทำไมต้องเป็นโมเดล “ขนาดใหญ่”?



**Larger Model =
(likely) Greater Capability**

The Modern AI Landscape

Closed Models



GPT-5.2



Claude Opus 4.6



Grok 4



Gemini 3

Open Models



GPT-OSS



Llama 4



Gemma 3



Mixtral



DeepSeek-V3.2



GLM-5



Qwen 3



Typhoon 2.5



Pathumma



OpenThaiGPT 1.6

Others:

- KhanomTanLLM
- WangChanGLM
- SeaLLM-7B
- SEA-LION-7B

Latest Update: 2026-02-20

The Modern AI Landscape

Closed Models



GPT-5.6



Claude Fable 5.1



Grok 4.5



**Gemini 3.6
Flash**

Open Models



GPT-OSS



Llama 4



Gemma 4



Mixtral



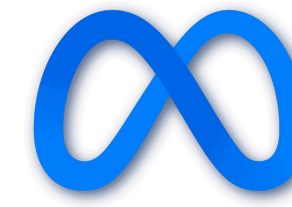
DeepSeek-V4



GLM-5.3



Qwen3.8



Muse Glimmer



Typhoon 2.5



Pathumma



OpenThai 2.0

Latest Update: 2026-08-02

Lab #1 เริ่มต้นใช้งาน LLM ด้วย LangGraph

เป้าหมาย

ทำความเข้าใจแนวคิดพื้นฐาน ได้แก่

1. **Prompt** ส่ง Prompt และรับคำตอบจาก LLM ผ่าน LangGraph
2. **Input/Output Tokens**: ตรวจสอบจำนวน Input, Output และ Thinking tokens
3. **Thinking Mode** เปิดและปิด Reasoning/Thinking mode
4. **Context Window**
5. **Max Token** จำกัดความยาวของคำตอบด้วย `max\_tokens`
6. **Finish Reason** ตรวจสอบสาเหตุที่โมเดลหยุดสร้างคำตอบผ่าน `finish\_reason`
7. **Temperature** ค่าที่ควบคุมความสุ่มของคำตอบ

+ ตัวอย่าง การสร้างแชทบอทแบบง่ายๆ ด้วย LangGraph



[Special thanks to Freepik on Flaticon for the icon](#)

Retrieval Augmented Generation 101



ข้อจำกัดของ LLMs



ขาดความรู้เฉพาะทาง

Lack of domain-specific knowledge



ไม่สามารถปรับปรุงข้อมูล
ความรู้ให้เป็นปัจจุบัน

Cannot real-time
knowledge updates



หลอน
(สร้างข้อมูลขึ้นมาเอง)

Hallucinations

3 แนวทางการแก้ไขปัญห

แนวทางที่ 1 Prompt + Context Engineering



Normal Prompt

แมวกินช็อกโกแลตได้ไหม??

Prompt with Context Engineering

แมวกินช็อกโกแลตได้ไหม??

โดยอ้างอิงคำตอบ จากเอกสารต่อไปนี้ [แนบไฟล์ คู่มือดูแลแมว]

คือ เพิ่มบริบทที่เกี่ยวข้องไปใส่ใน prompt โดยตรง

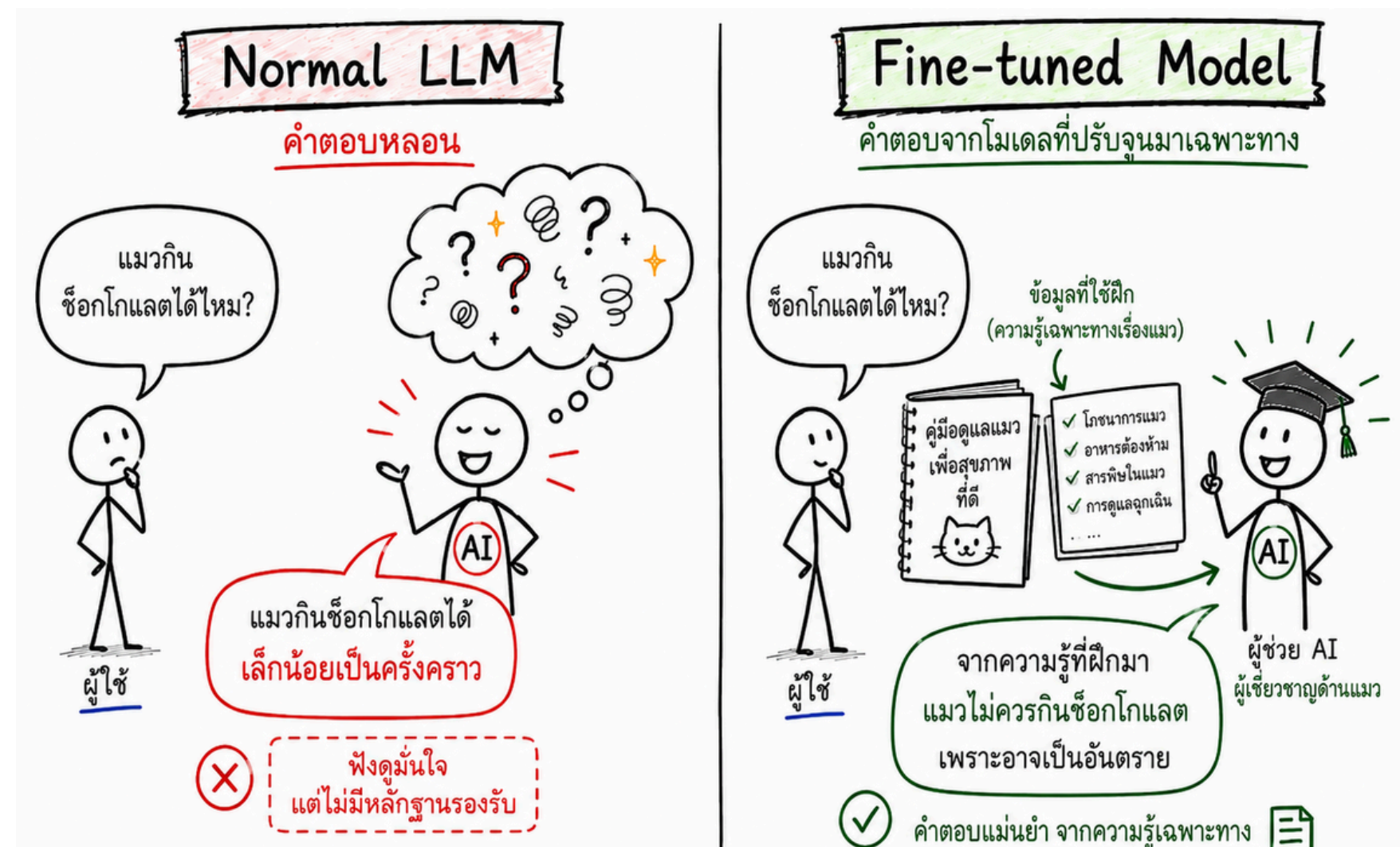
- ง่าย ทำได้ทันที
- แต่ ผู้ถาม ต้องแนบเอกสารที่เกี่ยวข้องด้วยตนเอง
- ต้นทุนสูง ถ้าเอกสารที่เกี่ยวข้องมีจำนวนมาก

3 แนวทางการแก้ไขปัญห

แนวทางที่ 2 Fine-tuning

คือ สอนโมเดลด้วยความรู้ที่ต้องการ เพื่อปรับพฤติกรรมของโมเดลแบบถาวร

- ใช้ต้นทุนสูง
 - ต้องใช้ข้อมูลจำนวนมาก
 - ต้องใช้ทรัพยากรคอมแรงๆ
- ต้องใช้เวลาในการสอน
- มีความเสี่ยงที่จะ performance ด้านอื่นๆ แย่ลง
- อัปเดตข้อมูลเพิ่มเติมไม่ได้



3 แนวทางการแก้ไขปัญห

แนวทางที่ 3 RAG

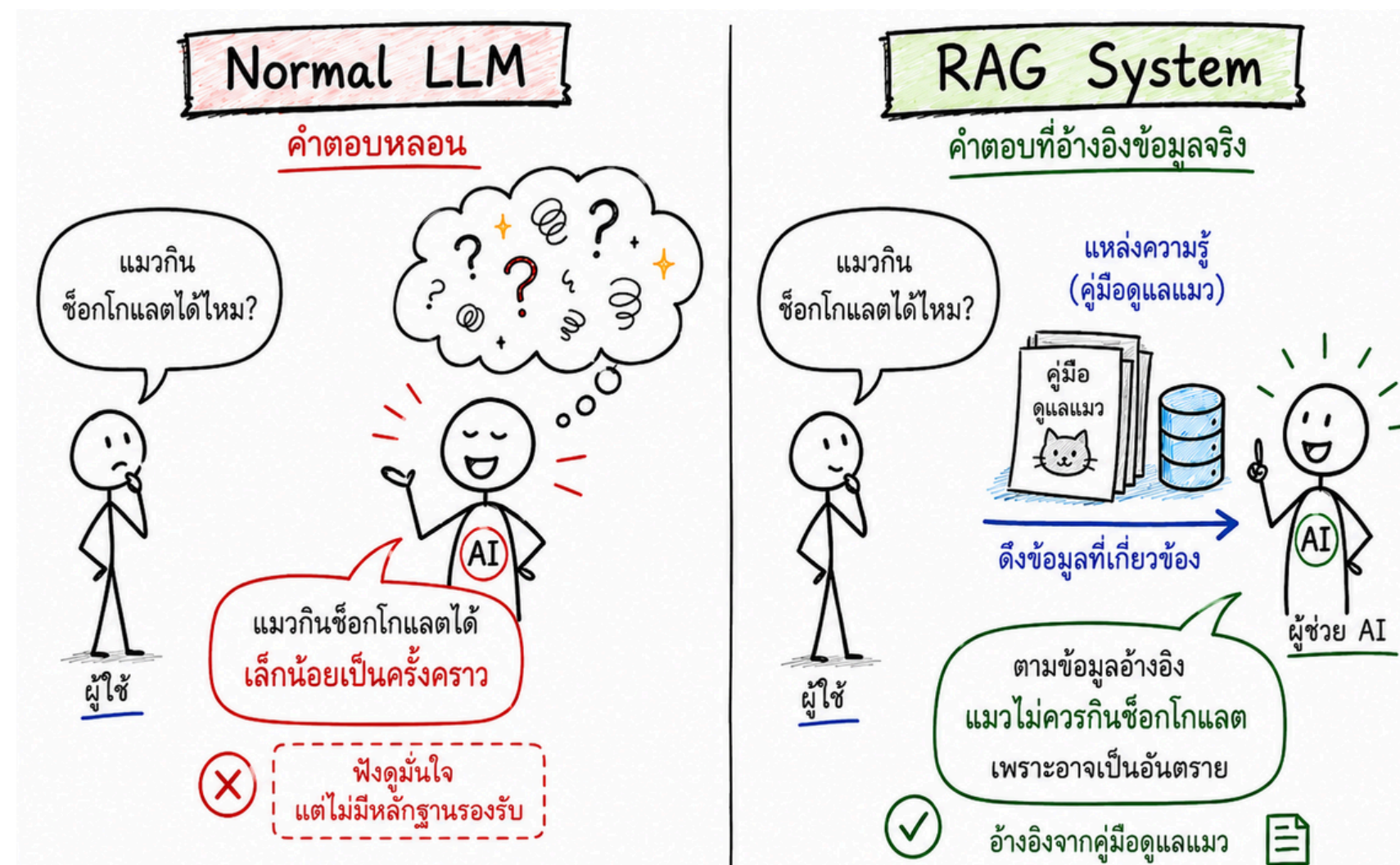
RAG = Context Engineering แบบ Advance

คือ การเพิ่มบริบทที่เกี่ยวข้องไปใส่ใน prompt โดยตรง แต่เลือกเฉพาะข้อมูลที่เกี่ยวข้องกับคำถามเท่านั้น

RAG = Retrieve + Augmented + Generation

“RAG is an AI framework for improving the quality of LLM-generated responses by grounding the model on external sources of knowledge”

-- [Stryker et al. \(2026\)](#)

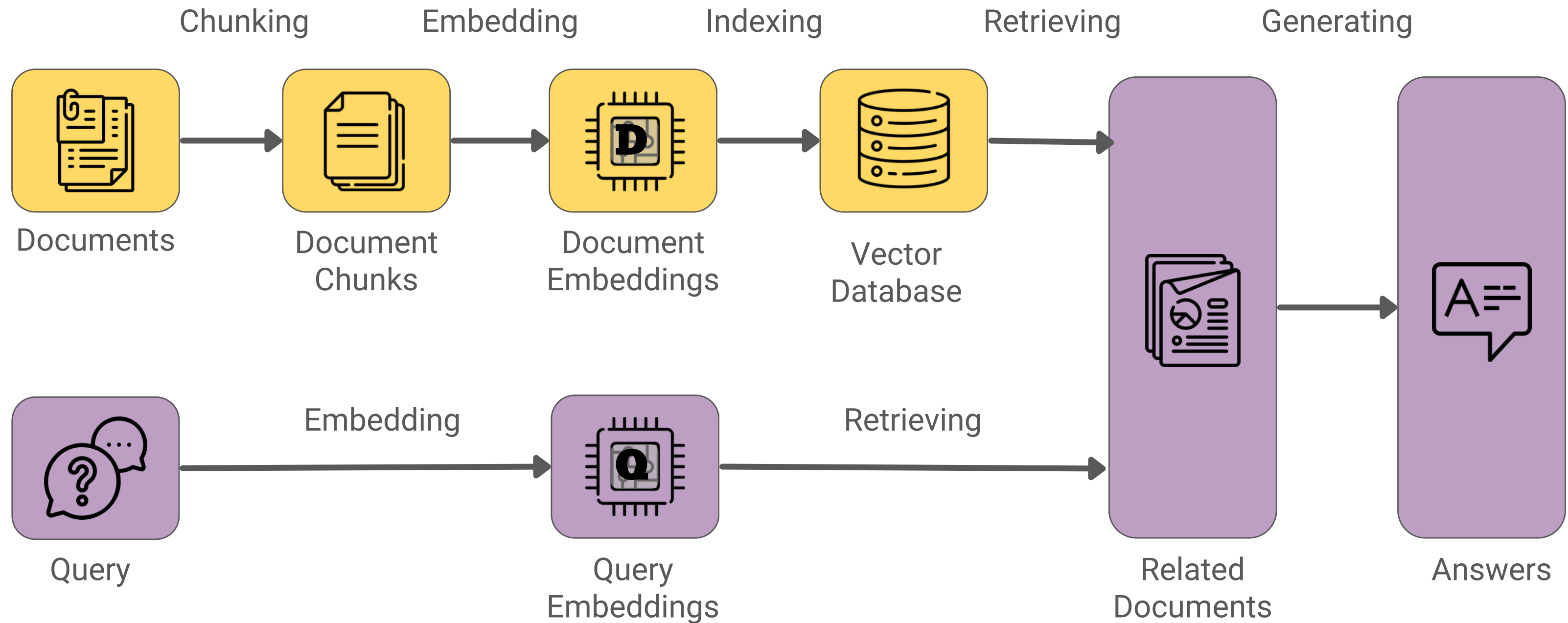


RAG vs Fine-Tuning

[Gao et. al, \(2023\)](#).

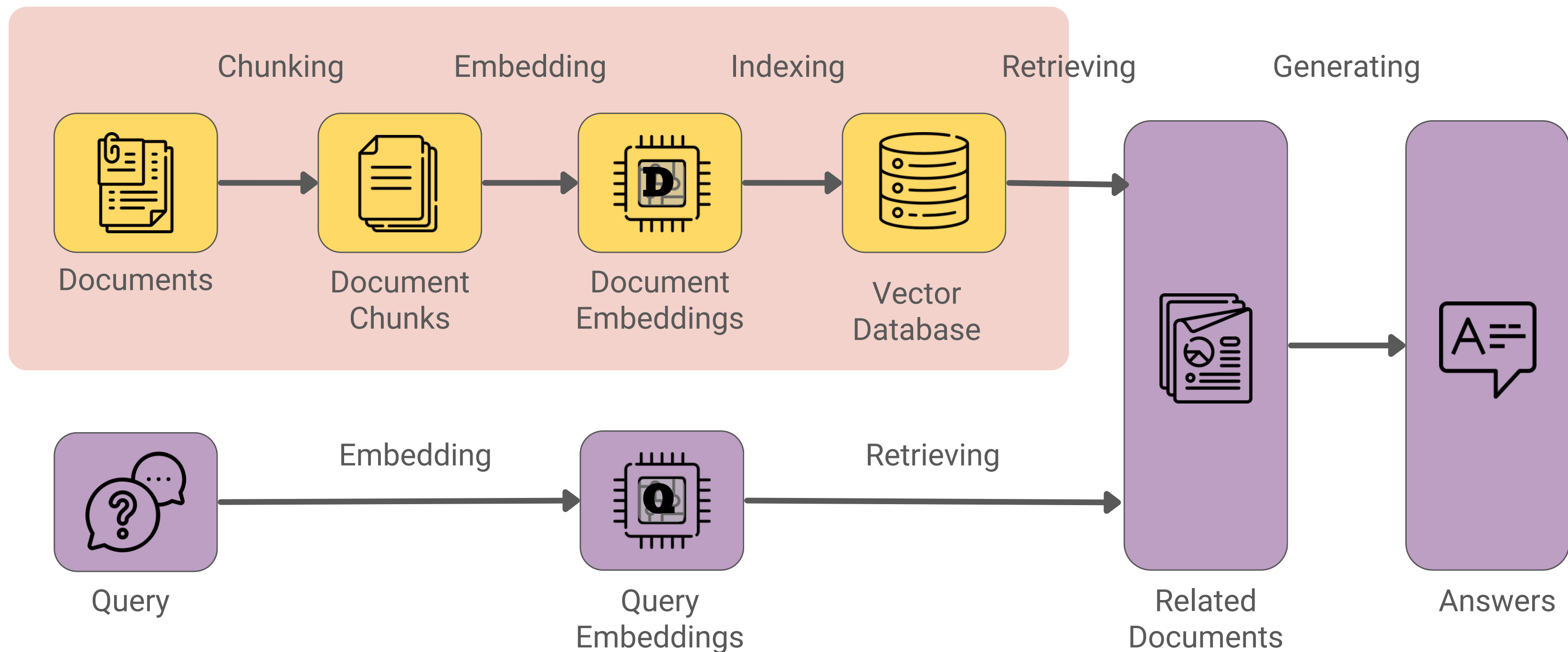
	RAG	Fine-tuning
Knowledge Update	Can directly updates	Require retraining
Model Customization	Difficult to fully customize model behavior or writing style	Allows adjustments of LLM behavior, writing style, or specific domain knowledge
Interpretability	Answers can be traced back to specific data sources	Not always clear why the model reacts a certain way
Ethical and Privacy Issues	Arise from from external databases.	Arise due to sensitive content in the training data

RAG Pipeline



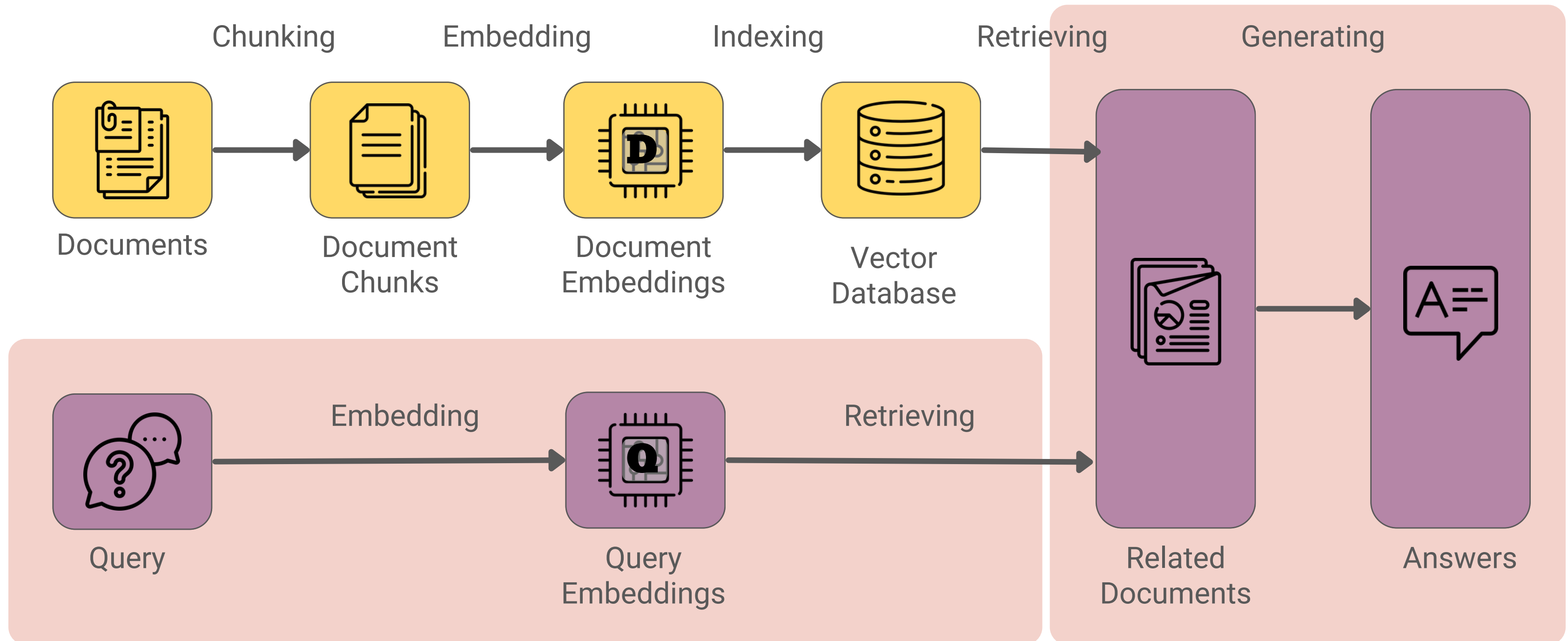
RAG Pipeline

ช่วงการเตรียมข้อมูล

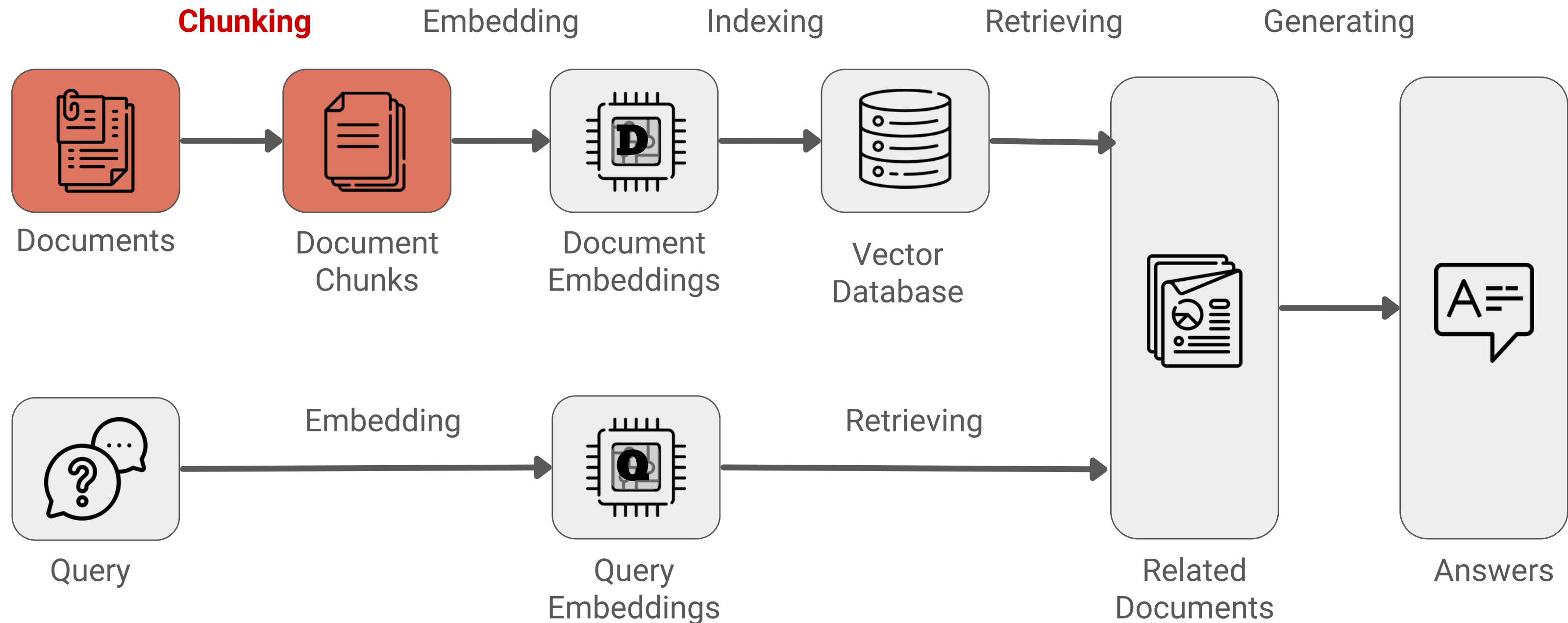


RAG Pipeline

เกิดขึ้นช่วงที่มีคนถามคำถาม



Step 1: Chunking



Step 1: Chunking

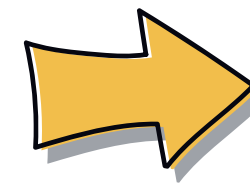
แมวพันธุ์ Russian Blue ถือเป็นแมวเก่าแก่ที่มีประวัติยาวนานพันธุ์หนึ่งเลยทีเดียว

หลายคนเชื่อว่าพวกมันมีต้นกำเนิดจาก Archangel Isles ใน Northern Russia ที่ซึ่งมีอากาศหนาวตลอดทั้งปี มันจึงมีขนที่หนาและนุ่มเป็นพิเศษ

นอกจากนี้ยังมีข่าวลือว่า บรรพบุรุษของ Russian Blue มาจากแมวของเหล่าผู้คนชั้นสูงในรัสเซีย



แมวพันธุ์ Russian Blue ถือเป็นแมวเก่าแก่ที่มีประวัติยาวนานพันธุ์หนึ่งเลยทีเดียว



หลายคนเชื่อว่าพวกมันมีต้นกำเนิดจาก Archangel Isles ใน Northern Russia ที่ซึ่งมีอากาศหนาวตลอดทั้งปี มันจึงมีขนที่หนาและนุ่มเป็นพิเศษ



นอกจากนี้ยังมีข่าวลือว่า บรรพบุรุษของ Russian Blue มาจากแมวของเหล่าผู้คนชั้นสูงในรัสเซีย

Step 1: Chunking

Chunking = การแบ่งเอกสารเป็นส่วนๆ แต่ยังคง **semantically independent**

- **เพิ่มความแม่นยำของการค้นคืนข้อมูล**
 - Chunk ที่มีขนาดเล็ก => เนื้อหาที่เฉพาะเจาะจงมากขึ้น => ค้นหาด้วย Similarity Search มีความแม่นยำยิ่งขึ้น
- **ลดบริบทที่ไม่เกี่ยวข้อง**
 - ประหยัดจำนวน Token => ลดทั้งค่าใช้จ่ายและเวลาในการประมวลผล
- **รองรับเอกสารขนาดใหญ่**
 - รองรับเอกสารที่ใหญ่เกินกว่าจะนำมา Embed (ขึ้นตอนถัดไป)
 - รองรับเอกสารที่ใหญ่เกินกว่าขนาด Context ของ LLM



Step 1: Chunking

Chunking = การแบ่งเอกสารเป็นส่วนๆ แต่ยังคง **semantically independent**

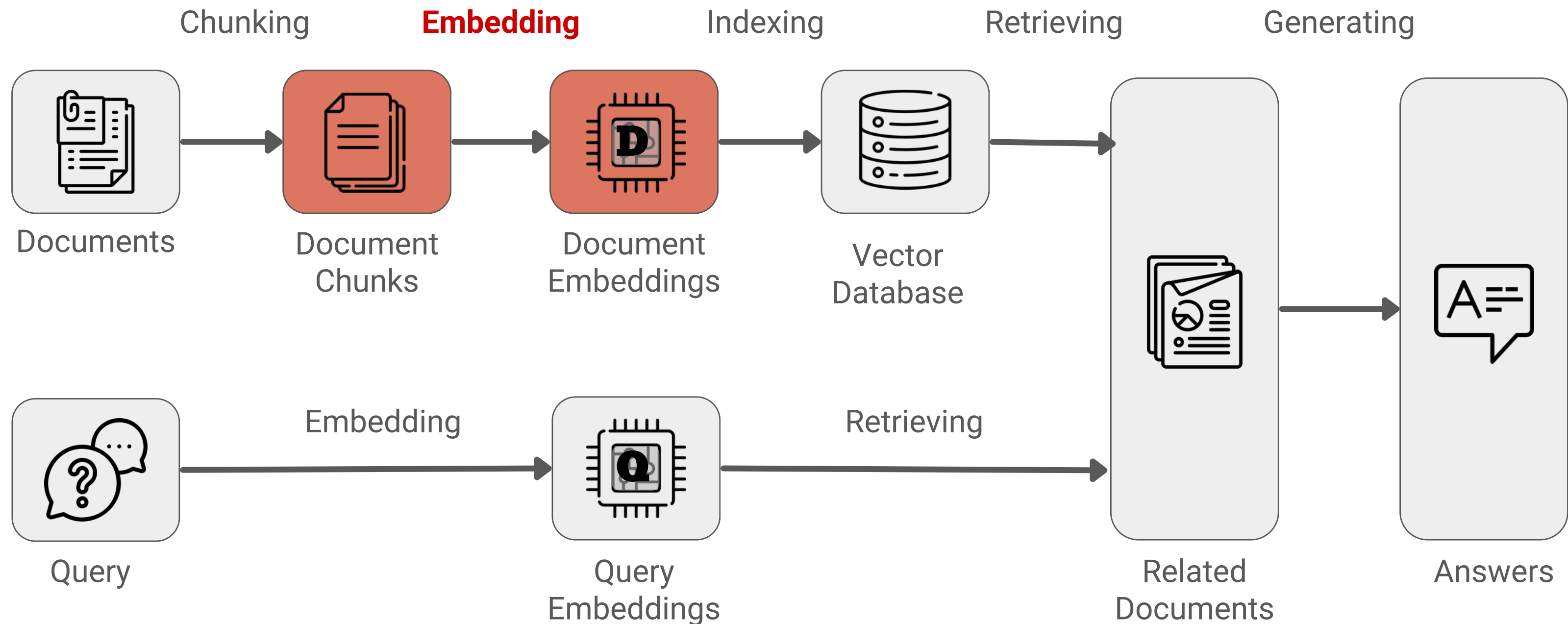
- **Fixed-size Chunking**
 - แบ่งข้อความตามจำนวนตัวอักษร หรือ Token ที่กำหนด
 - ง่ายและเร็ว
- **Sliding Window / Overlapping Chunking**
 - แบ่งให้แต่ละ chunk มีข้อมูลซ้ำกับ chunk ข้างเคียง
 - ช่วยลดโอกาสที่ข้อมูลสำคัญจะถูกตัดตรงบริเวณขอบของ chunk
- **Sentence-based / Paragraph-based Chunking**
 - แบ่งตามขอบเขตของประโยค/ย่อหน้า
 - อ่านและตีความได้เป็นธรรมชาติกว่า fixed-size chunking
- **Semantic Chunking**
 - แบ่ง chunk เมื่อพบว่าเนื้อหาเริ่มเปลี่ยนประเด็น
 - ซับซ้อน เพราะต้องตรวจสอบประเด็นที่กำลังเขียนถึง

Chunking Size = ??

- Chunk too small → loses context
- Chunk too large → introduces noise

<https://aclanthology.org/2024.tacl-1.9/>

Step 2: Embedding



Step 2: Embedding

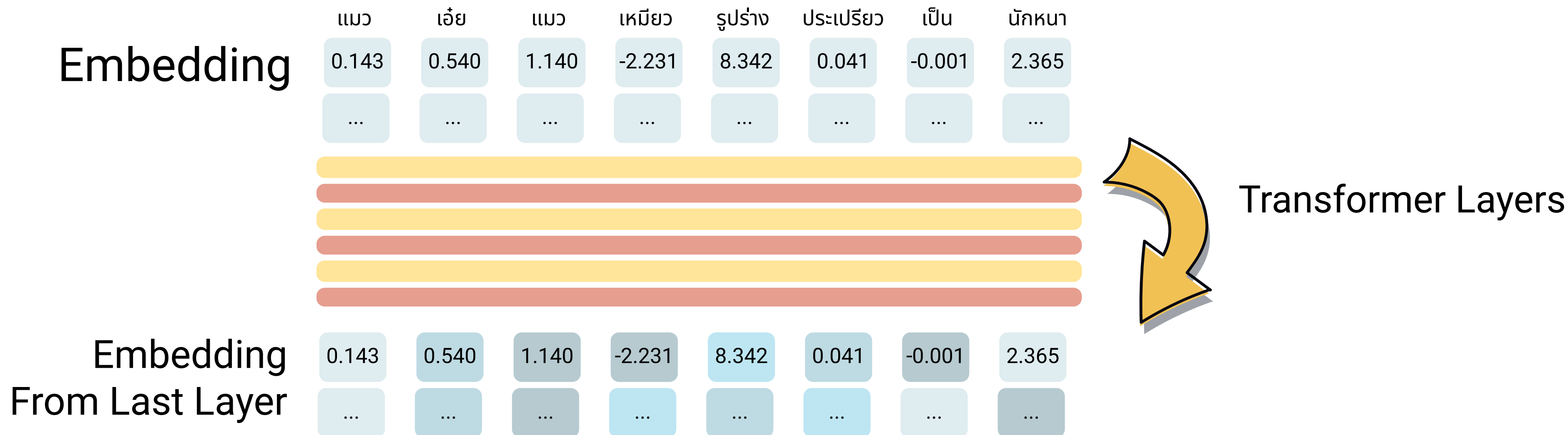
Embedding = การแปลงข้อความในแต่ละ Chunks เป็น เวกเตอร์ เพื่อไปใช้ใน semantic search

Tokens	แมว	เ้อย	แมว	เหมียว	รูปร่าง	ประเปรี้ยว	เป็น	นักหนา	Embedding
Embedding	0.143	0.540	1.140	-2.231	8.342	0.041	-0.001	2.365	
	0.324	-3.424	-0.233	-0.324	0.324	2.024	0.292	-1.000	
	-1.233	2.253	-1.233	1.030	-1.033	0.292	0.284	0.324	
	...	...	...	...	...	...	...	...	
	5.293	-2.293	0.243	8.250	-7.221	-1.284	0.050	9.383	



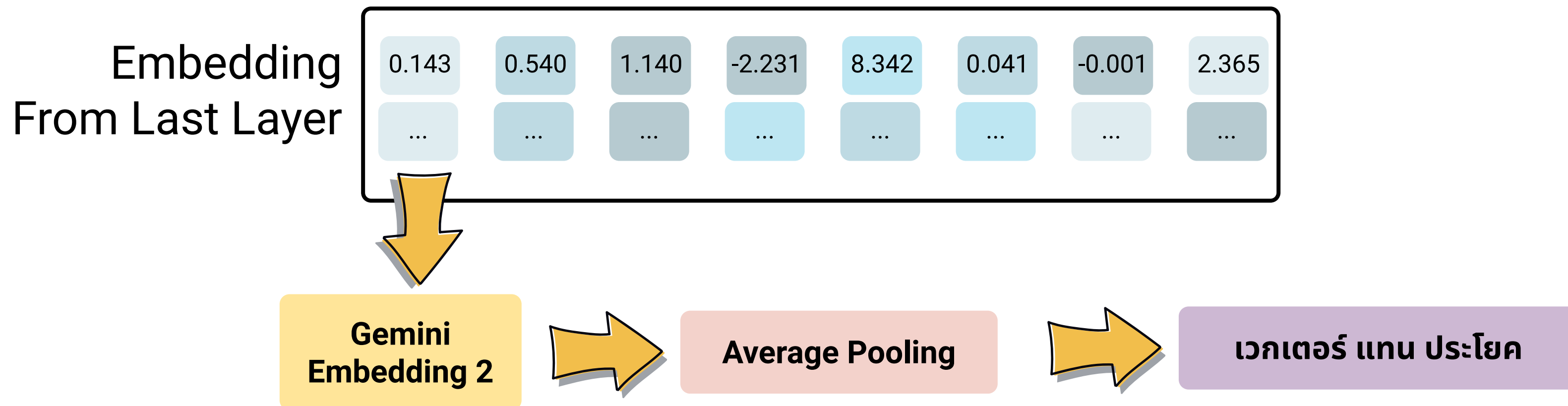
Step 2: Embedding

Embedding = การแปลงข้อความในแต่ละ Chunks เป็น เวกเตอร์ เพื่อไปใช้ใน semantic search



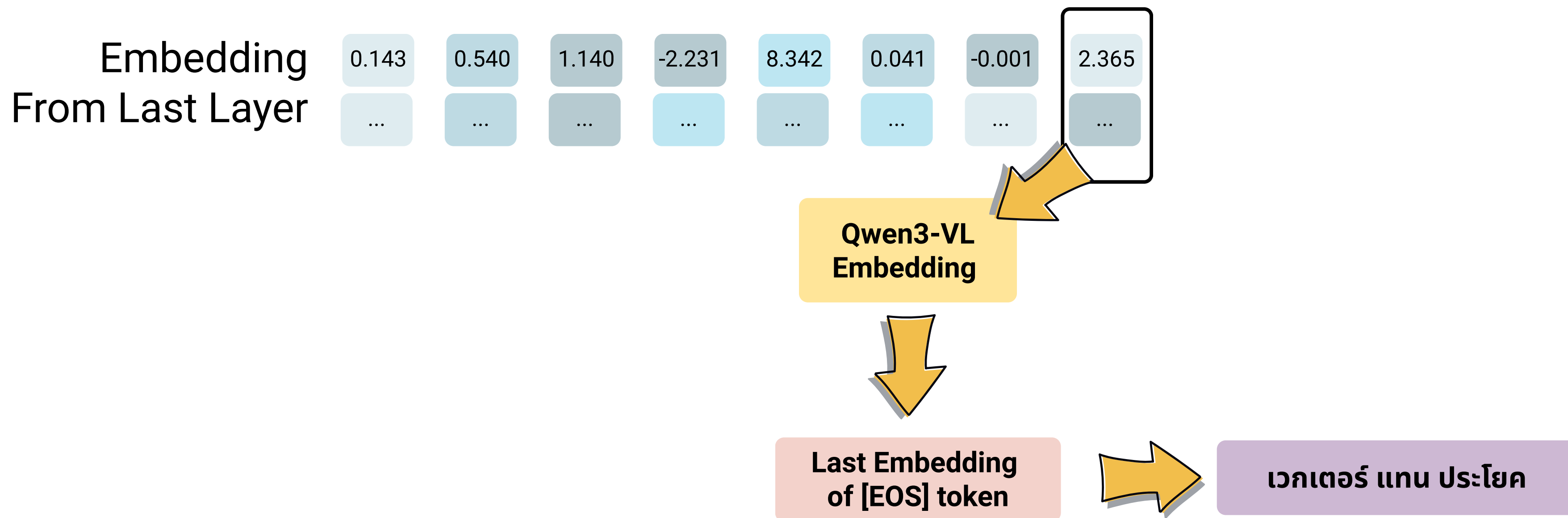
Step 2: Embedding

Embedding = การแปลงข้อความในแต่ละ Chunks เป็น เวกเตอร์ เพื่อไปใช้ใน semantic search



Step 2: Embedding

Embedding = การแปลงข้อความในแต่ละ Chunks เป็น เวกเตอร์ เพื่อไปใช้ใน semantic search

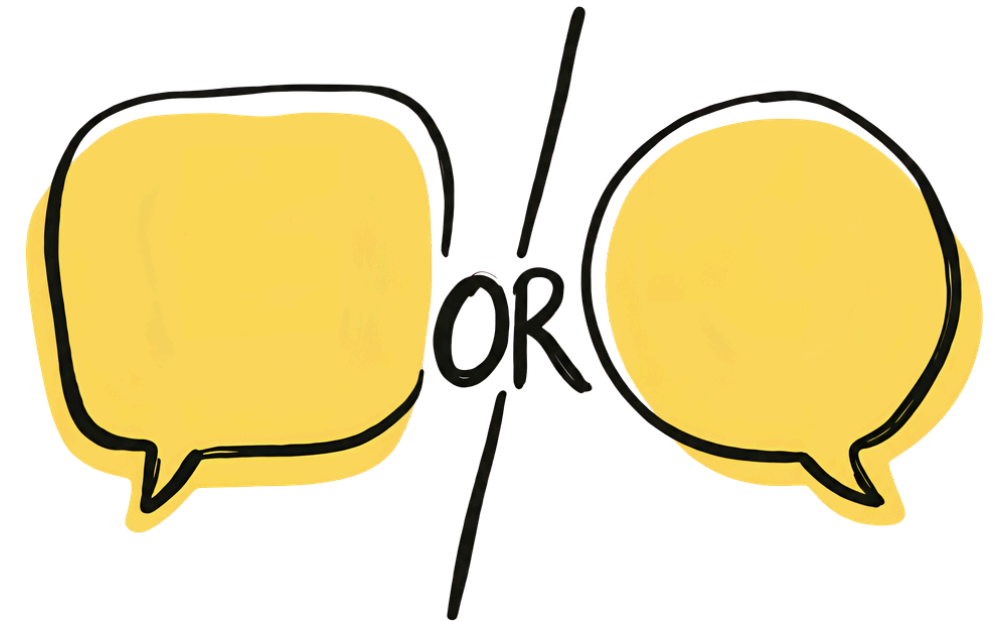


Step 2: Embedding

Embedding Models ต่างจาก Based Model ยังไง?

Embedding Models = Based Model + เทรนเพิ่ม เพื่อเพิ่มคุณสมบัติให้ embedding

- (InfoNCE) Contrastive Loss
- Matryoshka Representation Learning (MRL)
- Re-Ranking Capability

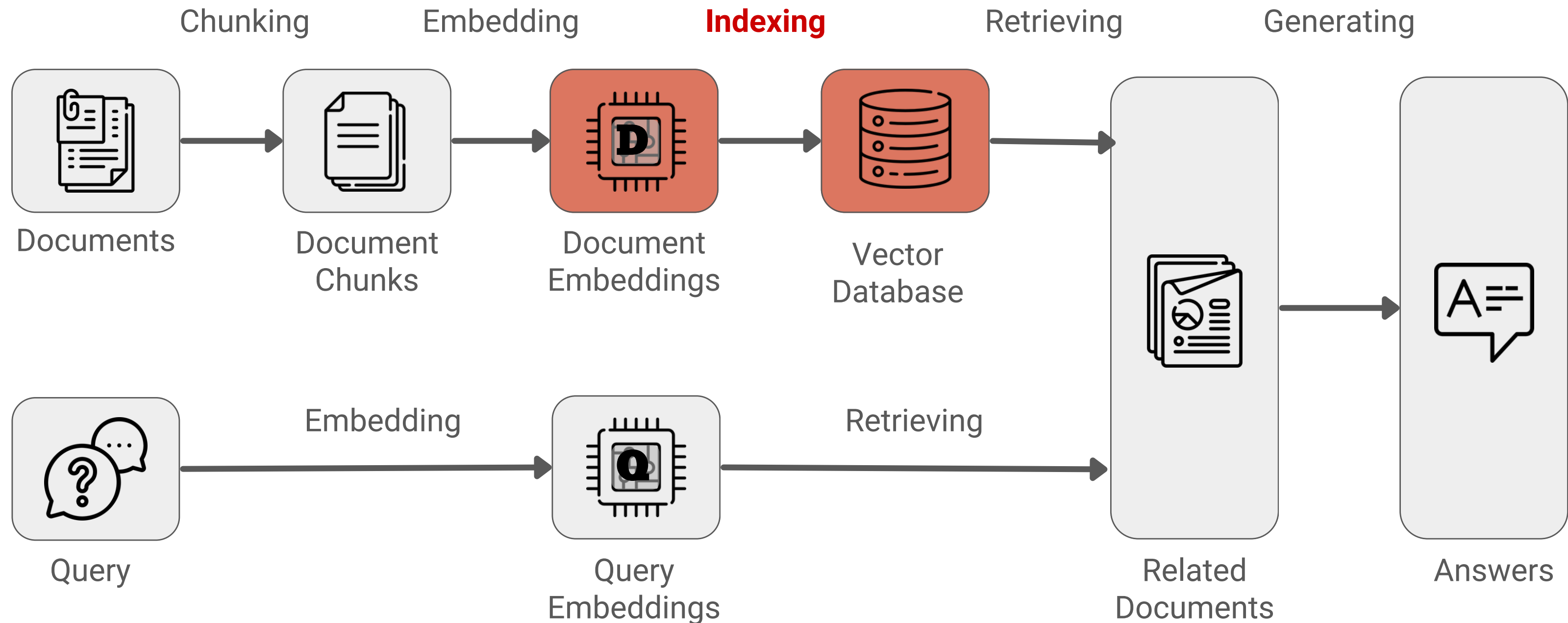


เลือก Embedding Models ยังไง?

เบื้องต้น สามารถดูประสิทธิภาพโมเดลได้ที่ [\[MTEB Leaderboard\]](#)

ศึกษาเพิ่มเติม: <https://qdrant.tech/articles/how-to-choose-an-embedding-model/>

Step 3: Indexing



Step 3: Indexing

Indexing = การจัดเก็บ Embedding และสร้างโครงสร้างสำหรับค้นคืนข้อมูลอย่างมีประสิทธิภาพ

ทำไมต้อง Indexing

- เพื่อสร้าง **Index Data Structure** ช่วยให้ค้นหา Candidate ที่ใกล้เคียงได้เร็วขึ้น
 - IVFPQ
 - HNSW
 - Annoy

สิ่งที่ต้อง Indexing

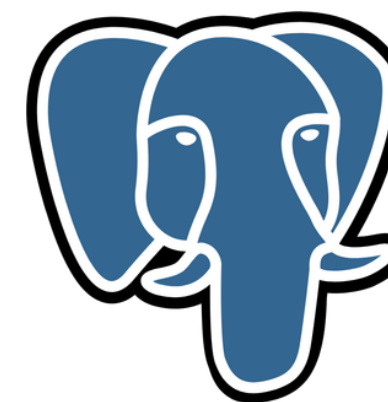
- Embeddings
- Metadata
 - สำหรับการ Filtering เพื่อเพิ่มประสิทธิภาพ
- Raw Data
 - ใช้อ้างอิง + ตรวจสอบ

Step 3: Indexing

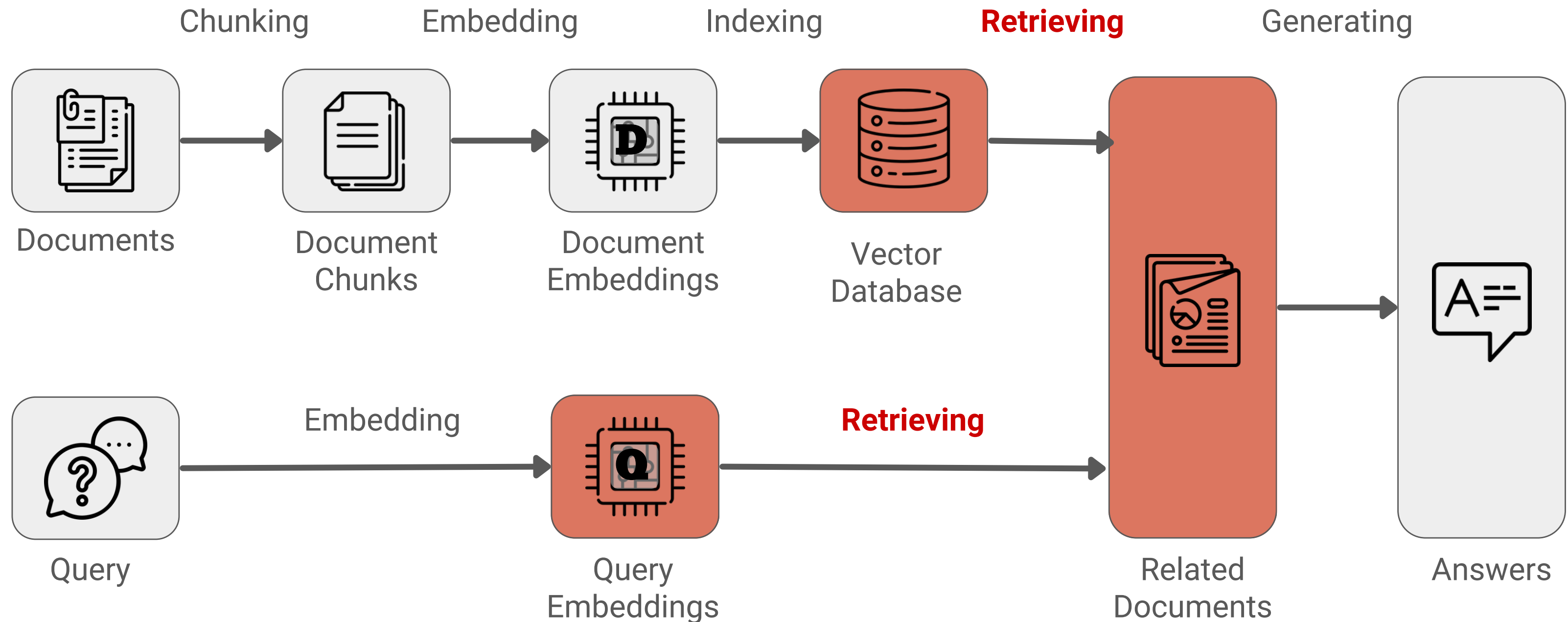
Indexing = การจัดเก็บ Embedding และสร้างโครงสร้างสำหรับค้นคืนข้อมูลอย่างมีประสิทธิภาพ

Vector Database

- Qdrant
- Milvus
- Elasticsearch
- Pgvector on Postgres



Step 4: Retrieving



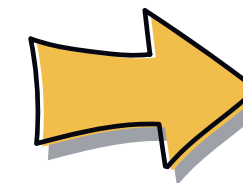
Step 4: Retrieving

Query: เพิ่งรับแมวมาใหม่แล้วแมวหลบใต้เตียงตลอด ควรทำอย่างไร?

Vector DB: แมวควรได้รับอาหารที่มีโปรตีนในปริมาณเหมาะสมตามอายุ น้ำหนัก และระดับกิจกรรมของร่างกาย

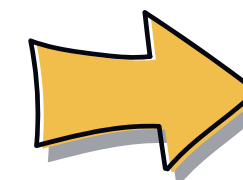
ควรจัดพื้นที่ปลอดภัยให้แมวใหม่ โดยวางอาหาร น้ำ และ กระบะทรายไว้ในบริเวณที่เงียบ ลดเสียงดังและการรบกวน

Embedded



0.143 0.540 ... 2.365

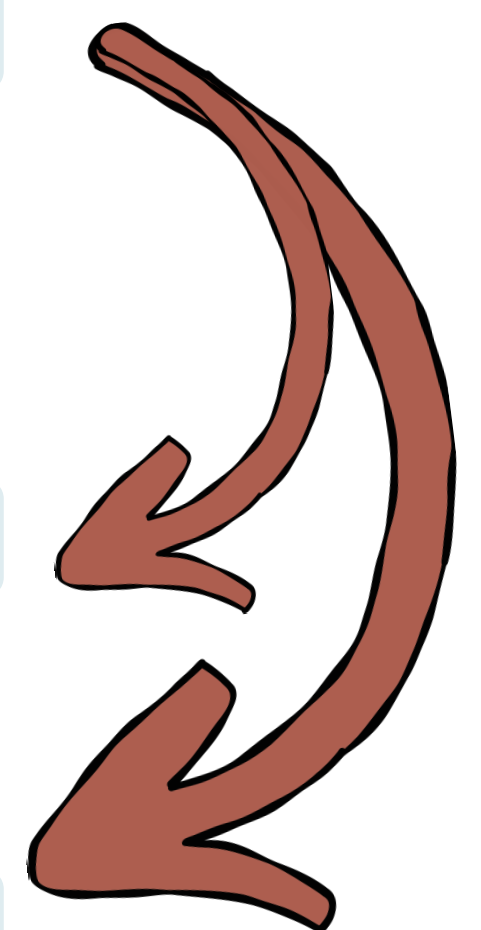
Embedded



-0.745 0.230 ... 0.002



0.204 -0.321 ... 0.238



Step 4: Retrieving

Retrieving = ค้นหา Embeddings ที่ความหมายใกล้เคียงกับคำถามที่สุด K ชั้น (Top-K)

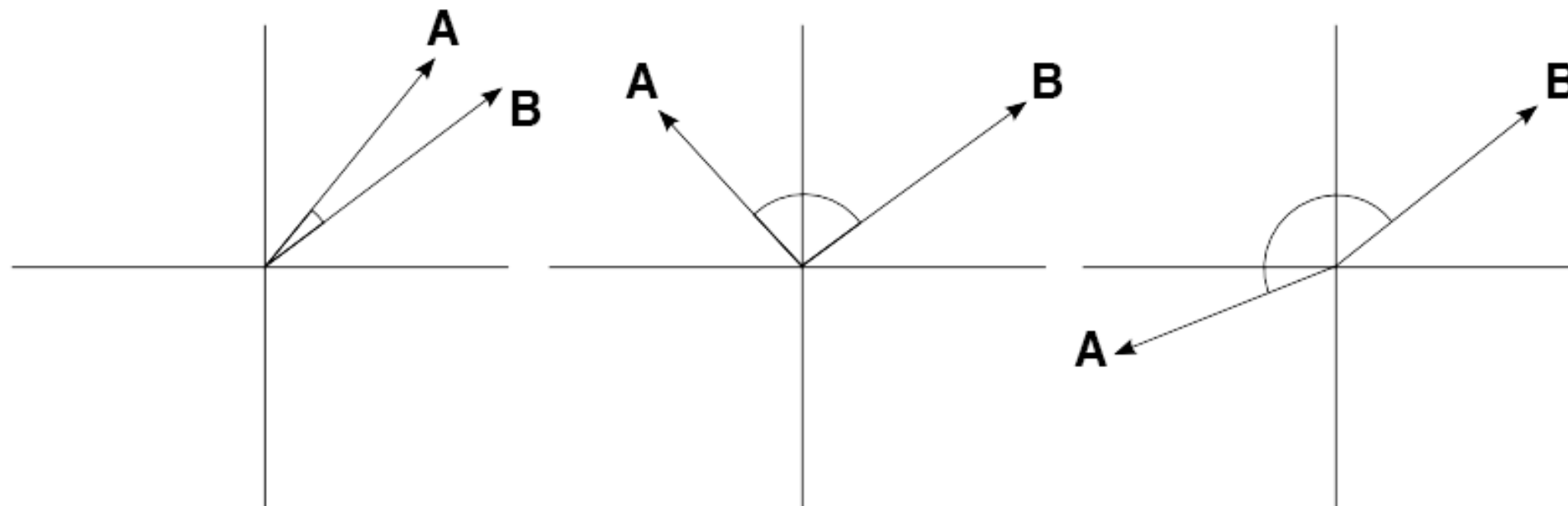
Similarity Metrics: **Cosine Similarity**

$$\text{Cosine Similarity} = \frac{A \cdot B}{\|A\| \|B\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$

Similar

Unrelated

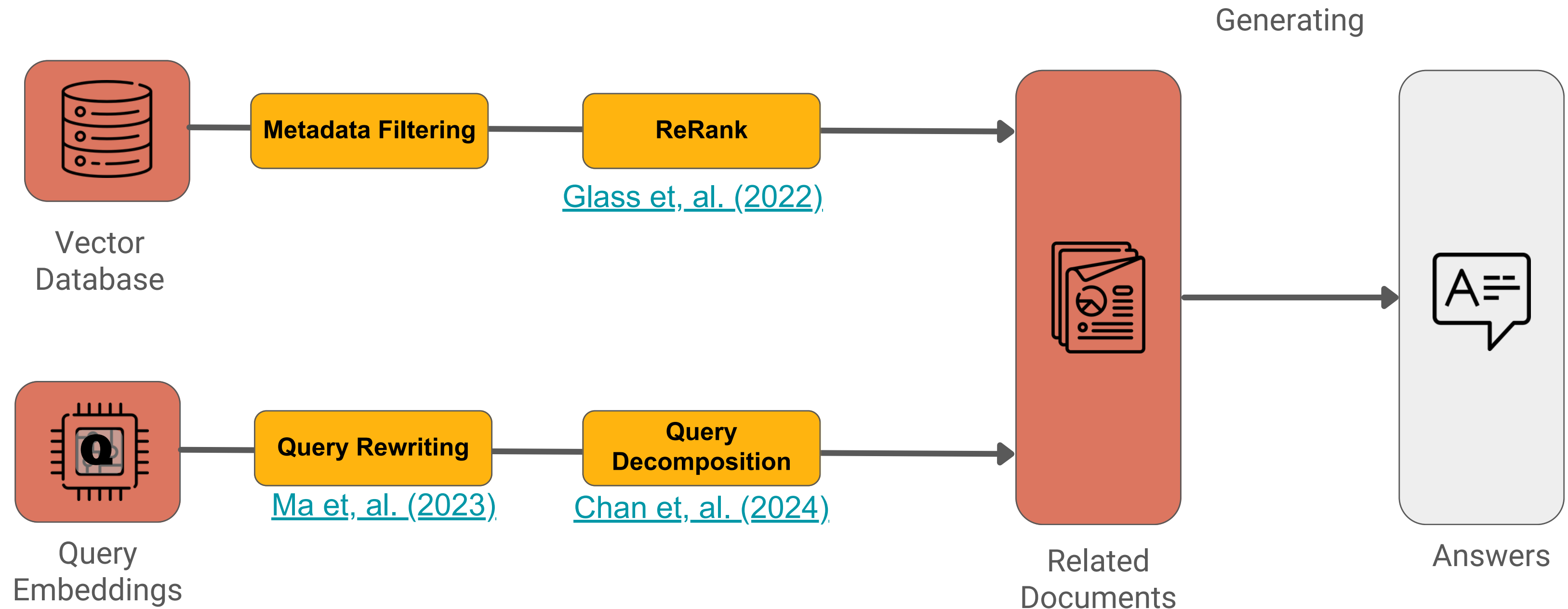
Opposite



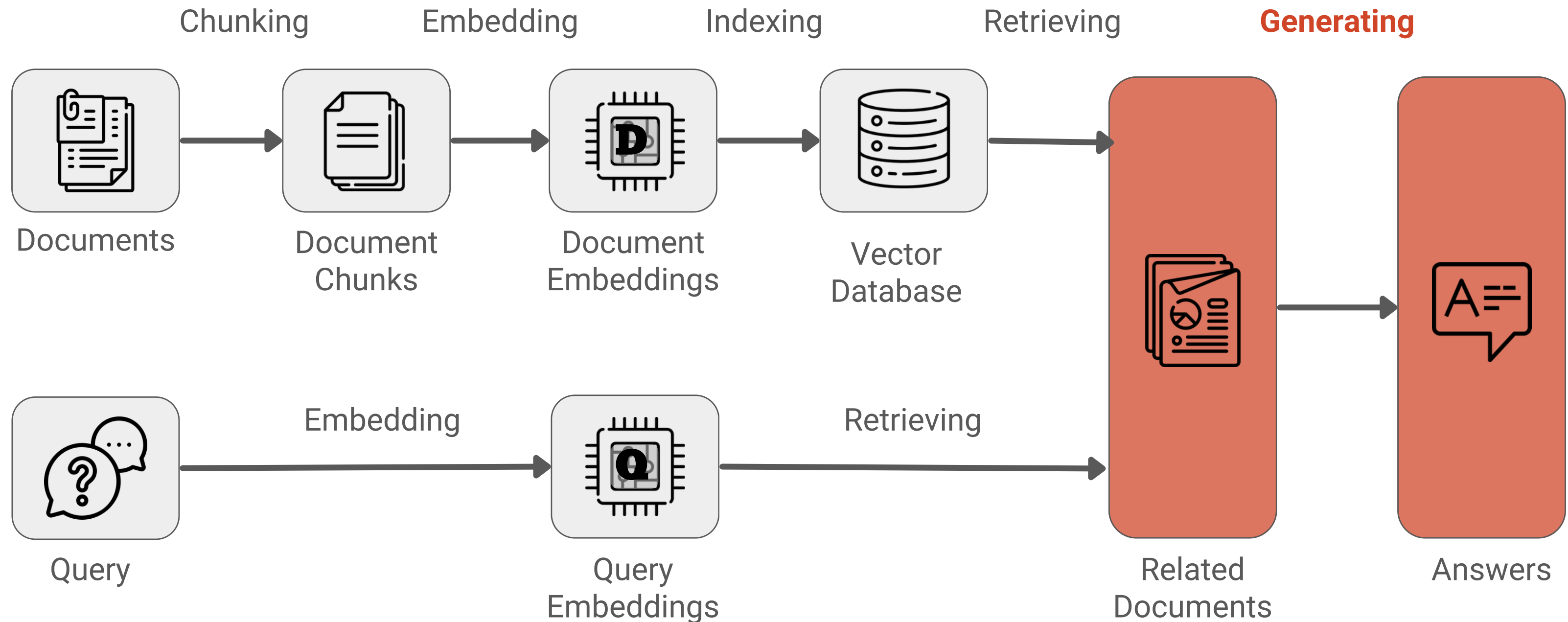
Assumption:

IF เวกเตอร์ A และ B มี cosine similarity สูงๆ
THEN เวกเตอร์ A และ B น่าจะเป็นสิ่งที่มีเนื้อหาคล้ายๆกัน

Step 4: (Advance) Retrieving



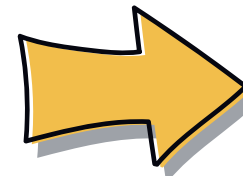
Step 5: Generating



Step 5: Generating

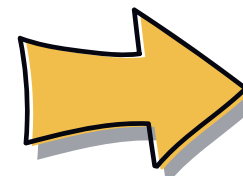
Related Documents

หลายคนเชื่อว่าพวกมันมีต้นกำเนิดจาก
Archangel Isles ใน Northern Russia



Query

Russian Blue มีต้นกำเนิดจากที่ไหน ?



Generator's Prompt

Instruction:

จงสรุปคำตอบใน Query โดยอ้างอิงบริบทจาก Context ต่อไปนี้

Context:

- หลายคนเชื่อว่าพวกมันมีต้นกำเนิดจาก Archangel Isles ใน Northern Russia
-

Query:

Russian Blue มีต้นกำเนิดจากที่ไหน ?

Step 5: Generating

Generating = การสั่งให้ LLM สร้างคำตอบโดยอาศัยข้อมูลที่ Retrieval

- **What if Retrieval does not contain the answer?**
- **Citation / Source Attribution**
- **Generation Quality**
 - ขึ้นอยู่กับ Retrieval Quality
 - Relevant : ตอบตรงคำถาม
 - Grounded : อ้างอิงจาก Context + ไม่สร้างข้อมูลเกินหลักฐาน
 - Traceable: ตรวจสอบ Source ได้

[Huang & Huang_\(2024\).](#)

Step 5: Generating + Customization

Generating = การสั่งให้ LLM สร้างคำตอบโดยอาศัยข้อมูลที่ Retrieval

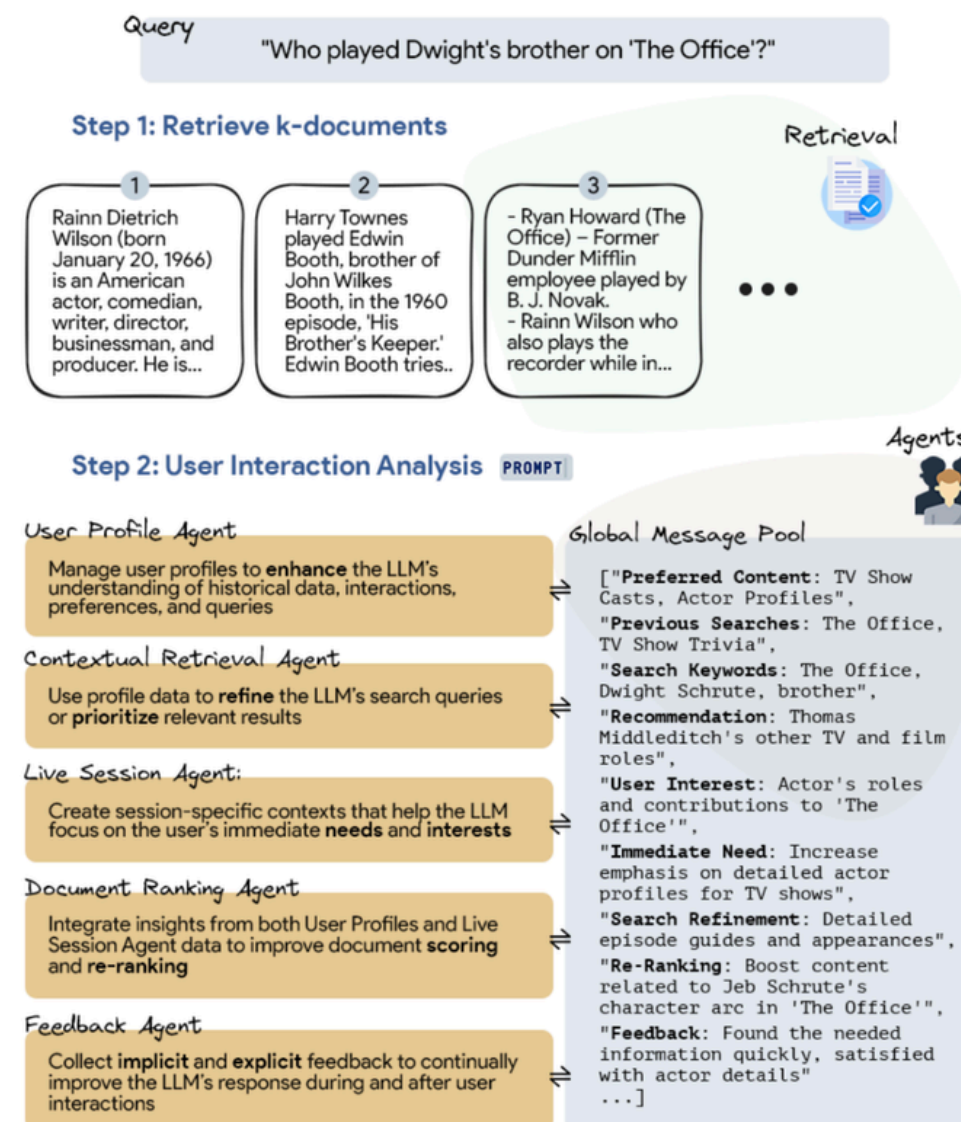
Persona

Chat History

Tone & Style

Level of Detail

Answer Policy / Guardrails



Step 3: Cognitive Dynamic Adaptation

Cognitive

Integrate insights derived from the User Interaction Analysis step into the system's retrieval strategies.

Agents can be used in two ways:
1. **Selectively**, by choosing one or multiple specific agents
2. **Collaboratively**, by utilizing content from the Global Message Pool

Chain-of-Thought

"To solve the problem, Please think and reason step by step, then answer"

Answer

The information in the passages includes varied mentions of characters related to Dwight Schrute, such as Ryan Howard and Harry Townes. However, these do not align directly with the query about Dwight's brother. The correct answer is **Thomas Middleditch**, who portrayed Dwight's brother Jeb Schrute, as verified by reliable passages.

"The episode guest stars Majandra Delfino, Blake Garrett Rosenthal, Thomas Middleditch, Matt Jones, and Tom Bower as members of Dwight's family... Middleditch

Each specialized agent worked collaboratively to analyze the query, refine search terms, and confirm that **Thomas Middleditch** played Dwight's brother on The Office.

- **User Profile Agent** identified the user's interest in 'The Office' and prioritized content related to Dwight Schrute's family.
- **Contextual Retrieval Agent** extracted critical terms and focused on guest cast lists for episodes featuring Dwight's family.
- **Document Ranking Agent** found passages stating that Thomas Middleditch portrayed Dwight's brother Jeb.
- **Live Session Agent** highlighted passages naming Dwight's relatives.
- **Feedback Agent** verified that the search results matched the query and confirmed Thomas Middleditch as Dwight's brother Jeb Schrute.

Lab #2 Embeddings

เป้าหมาย

ทำความเข้าใจแนวคิดพื้นฐาน เกี่ยวกับ embeddings

1. สร้าง Dense Embedding ด้วย LangChain
2. อธิบายความแตกต่างระหว่าง ****Sparse Embedding**** และ ****Dense Embedding****
3. คำนวณ ****Cosine Similarity**** ระหว่างข้อความสองข้อความ
4. คำฟ้อง การสะกดผิด และการแปลข้ามภาษา
5. (Optional) ลองใช้ FAISS



[Special thanks to Freepik on Flaticon for the icon](#)

Lab #3 Simple RAG

เป้าหมาย

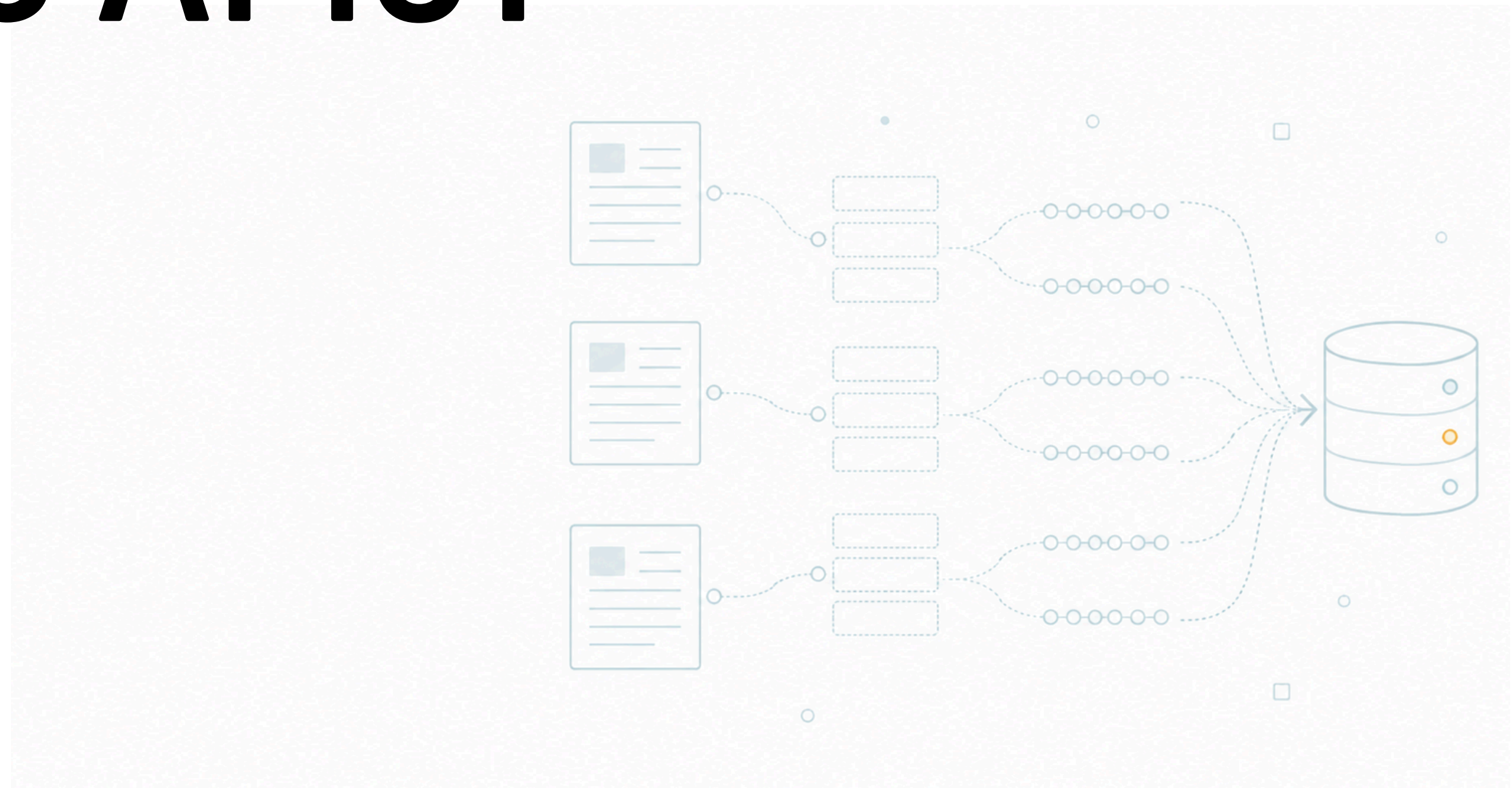
พัฒนาสร้างระบบ RAG (Retrieval-Augmented Generation) เป็นระบบแนะนำกระทู้ Pantip ที่เกี่ยวข้อง โดยอ้างอิงข้อมูล [\[krathu-500\]](#) และเปรียบเทียบ Retrieval 3 รูปแบบ

1. Simple Retrieval — Dense semantic search
2. Hybrid Retrieval — Dense retrieval + BM25
3. Retrieval + Reranking — Retrieval แบบคร่าว แล้วให้ LLM จัดอันดับใหม่ก่อนตอบ

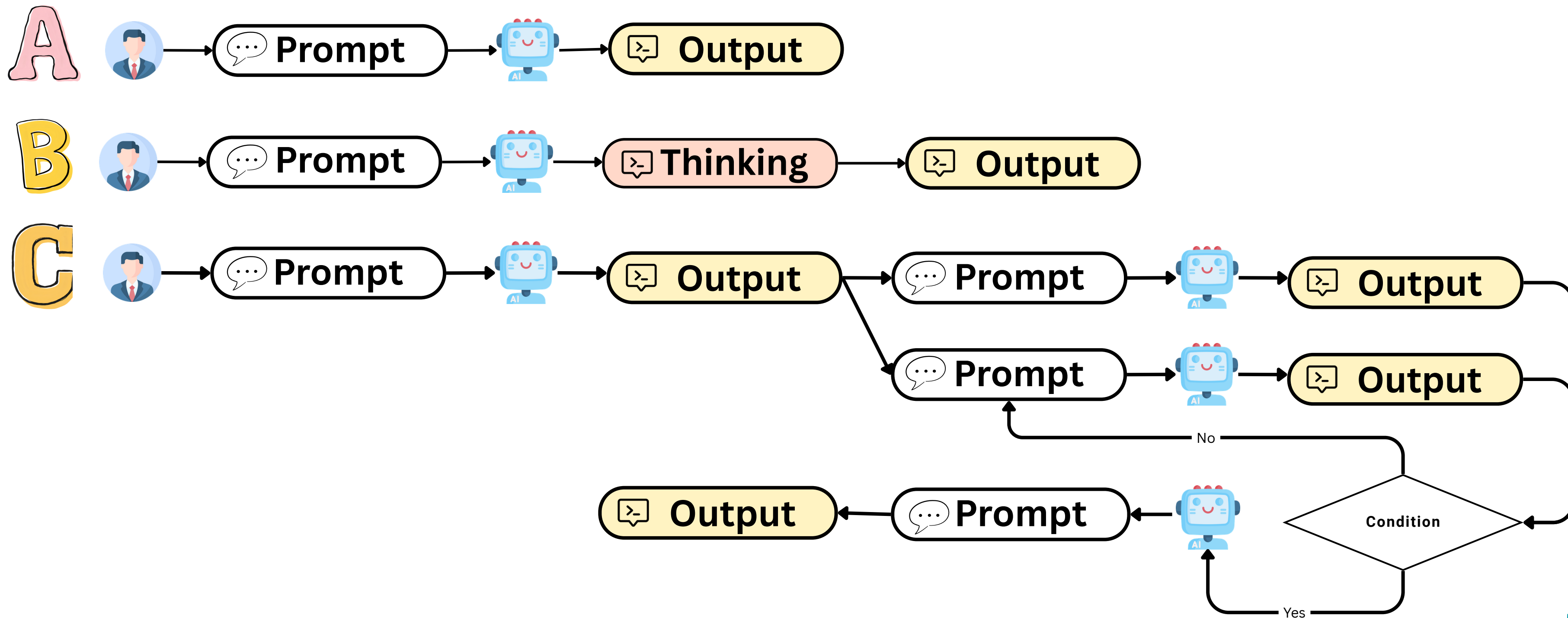


[Special thanks to Freepik on Flaticon for the icon](#)

Agentic AI 101



Which one is Agentic AI?



What is Agentic AI?

- **Def:** Agentic AI is “a system based on a foundation model that performs tasks potentially yielding artifacts based on natural user instructions, where the system is able to conduct and express complex reasoning including **planning** and **reflection** to solve tasks that require **interaction** with an environment or elaborate **tool use**”.

Planing

Reflection

Interaction

Tool use

Why Agentic AI?

J. Schneider, (2025).

Reasoning Test-time reasoning with Planning and Reflection (e.g., ChatGPT o1)	Multi-step reasoning with Chain-of-thought (e.g., ChatGPT 4.5)	No reasoning (e.g., ChatGPT 3.5)
I want to add two numbers using very basic addition. Tell me, what is 1+1?		
Reasoned about basic addition for a second 1 + 1 = 2. ✓	1 + 1 equals 2. ✓	1 + 1 equals 2. ✓
A banker is as old as his Ferrari will be when the banker is twice as old as the car was when the banker's age was half the sum of their present age. Take into account the financial crises of 2008. What is the age of banker and the car?		
Reasoned about banker's and car's age for 1m 5s Step 1 Analysis and Planning Tracking key events > Breaking down the events > Piecing together connections > Mapping future growth > Step 2 Solving Weighing puzzle clues > Evaluating age constraints > Step 3 Reflection and Verification Evaluating standard answers > Evaluating Banker = 48, Ferrari = 36. Verifying calculations > ✓	(Immediately respond) Step 1: Let banker's age = B , Ferrari's age = F . From the riddle, we get: $B = 2 \left(F - \frac{B - F}{2} \right)$ Step 2: Simplifying gives: $B = \frac{3F}{2} \Rightarrow B - F = \frac{F}{2}$ Step 3: So in $B - F$ years, Ferrari will be B years old — confirms the logic. Step 4: Trying $F = 18$ (hint from 2008 crisis), gives $B = 27$, which satisfies all conditions. Answer: Banker = 27, Ferrari = 18. ✗	(Immediately respond) Banker's age: 22 years and 8 months Ferrari's age: 17 years ✗

Traditional Generative AI

operates in a single pass
without the ability to
revise its output

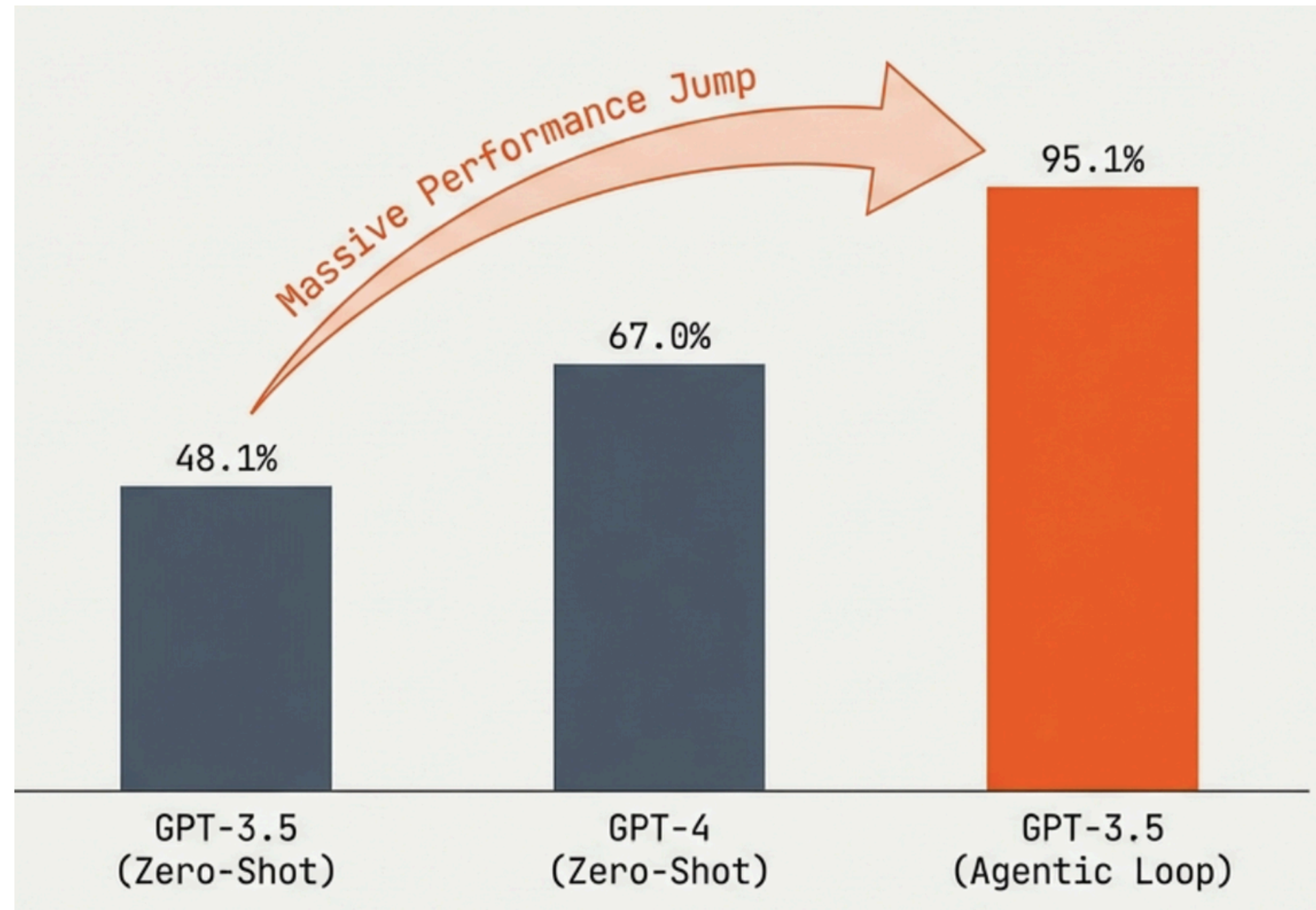
Agentic AI

uses **iterative planning, reasoning, and environmental interaction** to autonomously solve multi-step problems

Why Agentic AI?

HumanEval Coding Benchmark Performance

[A. Ng \(2024\)](#)



What is Agentic AI?



Planing

breaks down a large objective into smaller subtasks and decides the best sequence of steps to achieve its goal



Tool use

access to external functions to gather information and take real-world actions



Reflection

examines its own work, checks for errors, and uses self-generated feedback to improve its output

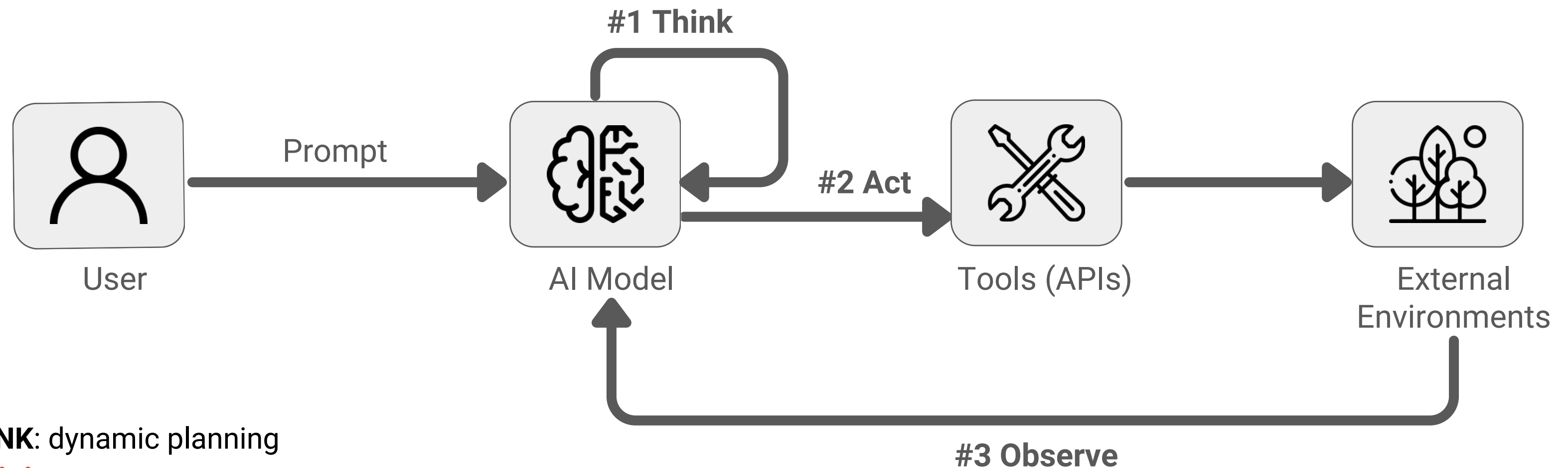


Interaction

collaborate with multiple agents adopting different expert roles to reach a better solution

[A. Ng \(2024\).](#)

ReAct Agent



- **THINK:** dynamic planning
 -  creating a 10-step plan at the start,
 -  plans its next move one step
- **ACT:** executes the next step in its plan
- **OBSERVE:** observes the new state
- ... Then it repeats until it has the final answer.

[Special thanks to Freepik on Flaticon for the icon](#)

<https://arxiv.org/abs/2210.03629>

ReAct Agent

(1) Hotspot QA

Question: Aside from the Apple Remote, what other device can control the program Apple Remote was originally designed to interact with?

(1a) Standard

Answer: iPod



(1b) CoT (Reason Only)

Thought: Let's think step by step. Apple Remote was originally designed to interact with Apple TV. Apple TV can be controlled by iPhone, iPad, and iPod Touch. So the answer is iPhone, iPad, and iPod Touch.

Answer: iPhone, iPad, iPod Touch



(1c) Act-Only

Act 1: Search[Apple Remote]

Obs 1: The Apple Remote is a remote control ...

Act 2: Search[Front Row]

Obs 2: Could not find [Front Row]. Similar: ...

Act 3: Search[Front Row (software)]

Obs 3: Front Row is a discontinued media center software ...

Act 4: Finish[yes]



(1d) ReAct (Reason + Act)

Thought 1: I need to search Apple Remote and find the program it was originally designed to interact with.

Act 1: Search[Apple Remote]

Obs 1: The Apple Remote is a remote control introduced in October 2005 by Apple ... originally designed to control the Front Row media center program ...

Thought 2: Apple Remote was originally designed to control the Front Row media center program. I need to search Front Row next and find what other device can control it.

Act 2: Search[Front Row]

Obs 2: Could not find [Front Row]. Similar: ['Front Row Seat to Earth', 'Front Row Motorsports', 'Front Row (software)', ...]

Thought 3: Front Row is not found. I need to search Front Row (software) .

Act 3: Search[Front Row (software)]

Obs 3: Front Row is a discontinued media center software ...

Thought 4: Front Row (software) is controlled by an Apple Remote or the keyboard function keys. So the answer is keyboard function keys.

Act 4: Finish[keyboard function keys]



[Yao et, al. \(2023\).](#)

Lab #4 ReAct Agent

เป้าหมาย

ทำความเข้าใจแนวคิดพื้นฐาน เกี่ยวกับ ReAct Agent

1. เข้าใจวงจร ReAct: Reason → Act → Observe
2. ให้ LLM ออกแบบ Plan ก่อนเริ่มทำงาน
3. ใช้ Tool calling เพื่อเรียกใช้เครื่องมือภายนอก
4. เพิ่ม Reflection เพื่อตรวจสอบหลักฐานและสิ่งที่ยังขาด
5. ป้องกัน agent loop ด้วย `max\_steps` และจัดการ tool error



[Special thanks to Freepik on Flaticon for the icon](#)

Agentic AI + RAGs

Limitation with Traditional RAG

- Similarity $\neq$ Relevance
- Chunking Can Destroy Knowledge Structure
- Fixed Top-K is an arbitrary assumption
- Multi-Hop Questions Require Iterative Search
- Traditional RAG cannot recover from bad retrieval

Agentic RAG describes how retrieval is controlled.

Vectorless RAG describes how evidence can be located.

AgenticRAG = RAG + Decision-Making Process

Traditional RAG

```
documents = retrieve(query)
answer = generate(query, documents)
```

No explicit state.
No corrective loop.
No dynamic tool choice.

Agentic RAG

```
while not enough_information():
    action = agent.plan(state)
    observation = execute(action)
    state.update(observation)
    answer = agent.generate(state)
```

 **Whether?**
Do I need retrieval?

 **How?**
How to get an answer

 **Where?**
Which knowledge source?

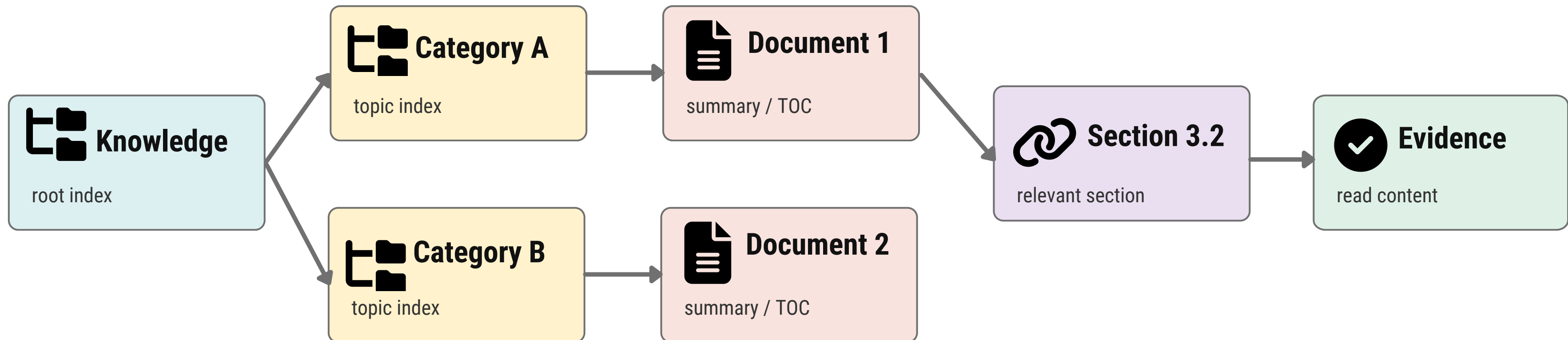
 **Stop?**
Is evidence sufficient?

Agentic RAG = retrieval + decision-making + feedback loop

From Agentic RAG to Vectorless RAG

Vectorless RAG

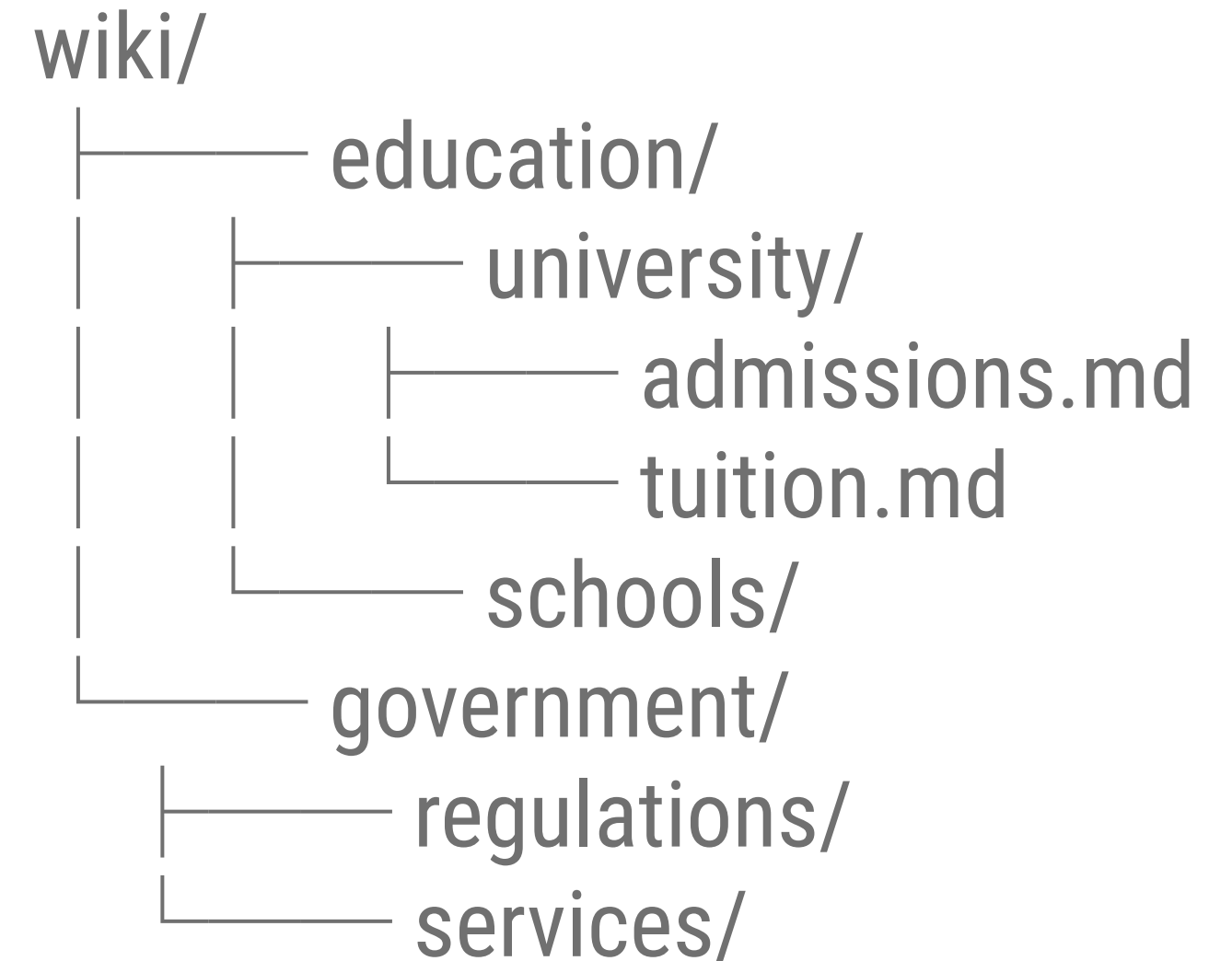
RAG in which dense-vector nearest-neighbor search is not the main mechanism used to locate evidence.



Agent Behavior: Read index → Choose branch → Inspect summary → Descend → Read evidence → Stop

Vectorless RAG

- LLM Wiki: https://github.com/nashsu/llm_wiki
 - see Andrej Karpathy's llm-wiki.md
- Google's Open Knowledge Format
 - <https://github.com/GoogleCloudPlatform/open-knowledge-format>
- Popularised by PageIndex
 - <https://github.com/VectifyAI/PageIndex>



Lab #5 Vectorless RAG

เป้าหมาย

พัฒนาสร้างระบบ RAG (Retrieval-Augmented Generation) เป็นระบบแนะนำกระตุ๋ Pantip ที่เกี่ยวข้อง โดยอ้างอิงข้อมูล [\[krathu-500\]](#) โดยอาศัยโครงสร้างข้อมูลในรูปแบบ Wiki (OKF) ร่วมกับ LLM ในการ Hierarchical Navigation แทนการค้นหา Vector Similarity Search



[Special thanks to Freepik on Flaticon for the icon](#)